

ISTQB · CTFL

Tester Certificado Nivel Básico

v 4.0

GUÍA PERSONAL DE ESTUDIO

6 Capítulos · Glosario · Errores Comunes

Tabla de Contenidos

1. Capítulo 1 — Fundamentos de las Pruebas

2. Capítulo 2 — Pruebas a lo Largo del CVDS

3. Capítulo 3 — Pruebas Estáticas

4. Capítulo 4 — Análisis y Diseño de Pruebas

5. Capítulo 5 — Gestión de las Actividades de Prueba

6. Capítulo 6 — Herramientas de Prueba

7. Glosario

8. Errores Comunes

Capítulo 1 - Fundamentos del Testing

La idea central: El testing es mucho más que hacer clic al azar y esperar que algo falle. Es una disciplina estructurada y profesional que ayuda a incorporar la calidad en el software — no solo a encontrar bugs al final.

Objetivos de Aprendizaje del Capítulo 1

LO	K	Tema
FL-1.1.1	K1	Identificar los objetivos típicos del testing
FL-1.1.2	K2	Diferenciar el testing del debugging
FL-1.2.1	K2	Ejemplificar por qué el testing es necesario
FL-1.2.2	K1	Recordar la relación entre el testing y el aseguramiento de la calidad
FL-1.2.3	K2	Distinguir entre error, defecto, fallo y causa raíz
FL-1.3.1	K2	Explicar los siete principios del testing
FL-1.4.1	K2	Resumir las diferentes actividades y tareas de testing
FL-1.4.2	K2	Explicar el impacto del contexto en el proceso de testing
FL-1.4.3	K2	Diferenciar el testware que soporta las actividades de testing
FL-1.4.4	K2	Explicar el valor de mantener la trazabilidad entre la base de pruebas y el testware
FL-1.5.1	K2	Explicar el rol de la psicología humana en el testing
FL-1.5.2	K1	Recordar las ventajas del enfoque de equipo completo
FL-1.5.3	K2	Distinguir los beneficios y desventajas de la independencia del testing

Consejo de examen: El Capítulo 1 es casi completamente K2 — enfócate en poder *explicar* y *distinguir*, no solo nombrar. Los únicos ítems K1 son FL-1.1.1, FL-1.2.2 y FL-1.5.2. No hay K3 en este capítulo.

¿Qué es el Testing, realmente? — 💡 Comprender (K2)

El testing es un conjunto de actividades para **descubrir defectos**, **evaluar la calidad** y **proporcionar información** para la toma de decisiones.

Nueve objetivos oficiales (K1 — conócelos todos): - Evaluar los productos de trabajo (requisitos, diseños, código) - Provocar fallos y encontrar defectos - Asegurar la cobertura requerida del objeto de prueba - Reducir el riesgo de calidad de software inadecuada - Verificar que los requisitos especificados hayan sido cumplidos - Verificar el cumplimiento de requisitos contractuales, legales y normativos - Proporcionar

información a los interesados para la toma de decisiones informadas - Generar confianza en la calidad del objeto de prueba - Validar si el objeto de prueba está completo y funciona según lo esperado

Conceptos erróneos comunes que hay que evitar: - El testing **no es solo** encontrar bugs — también genera confianza en que el software funciona - El testing **no es solo** ejecutar código — el testing estático (revisión de documentos, diseños) también cuenta - El testing cubre tanto la **verificación** ("¿lo estamos construyendo bien?") como la **validación** ("¿estamos construyendo lo correcto?")

Testing vs. Debugging — 💡 Comprender (K2)

Actividad	Quién la realiza	Qué hace
Testing	Testers	Encuentra fallos y defectos
Debugging	Desarrolladores	Encuentra la causa, la corrige, elimina el defecto

Proceso de debugging (3 pasos): (1) Reproducción del fallo, (2) Diagnóstico — encontrar la causa raíz, (3) Corrección del defecto. Tras la corrección: el **testing de confirmación** verifica que la corrección resolvió el fallo; el **testing de regresión** verifica que nada más se haya roto. Matiz del testing estático: el testing estático encuentra defectos directamente sin necesidad de ejecutar el código — no se necesita el paso de reproducción del fallo.

¿Por Qué es Necesario el Testing? — 💡 Comprender (K2)

El testing contribuye al éxito del proyecto de tres maneras distintas: - **Detección de defectos rentable** — eliminar defectos (debugging) es una actividad no de testing; el testing proporciona la evidencia que lo desencadena - **Evaluación de la calidad en las fases del SDLC** — evalúa directamente la calidad en varios puntos e informa las decisiones de lanzamiento - **Representación indirecta de los usuarios** — los testers aseguran que las necesidades del usuario se consideren durante todo el proyecto

La cadena error → defecto → fallo — 💡 Comprender (K2)

- **Error:** Un error humano (p. ej., un desarrollador malinterpreta un requisito)
- **Defecto:** La falla introducida por ese error en el código o documento
- **Fallo:** Comportamiento incorrecto cuando ese defecto es ejecutado
- **Causa raíz:** La razón subyacente por la que ocurrió el error

No todos los defectos causan fallos — algunos solo fallan bajo condiciones específicas, algunos pueden no causar nunca un fallo.

Testing vs. Aseguramiento de la Calidad (QA) — 🧠 Recordar (K1)

- El **testing** es **orientado al producto** y **correctivo** — encuentra defectos en lo que fue construido
- El **QA** es **orientado al proceso** y **preventivo** — mejora el proceso para prevenir defectos

Los 7 Principios del Testing — 💡 Comprender (K2)

#	Principio	Qué significa en la práctica
1	El testing muestra la presencia de defectos, no su ausencia	Pasar todas las pruebas no significa que no haya bugs
2	El testing exhaustivo es imposible	Usa el riesgo y las prioridades para enfocar los esfuerzos
3	El testing temprano ahorra tiempo y dinero	Cuanto más tarde se encuentra un defecto, más caro es corregirlo
4	Los defectos se agrupan	Un pequeño número de módulos contiene la mayoría de los defectos (Pareto)
5	Las pruebas se desgastan	Las mismas pruebas encuentran menos defectos nuevos — varíalas y actualízalas
6	El testing depende del contexto	Probar una aplicación bancaria es diferente a probar un juego
7	La ausencia de defectos es una falacia	Un sistema sin bugs que no cumple con las necesidades del usuario sigue siendo un fracaso

Actividades y Tareas del Testing — 💡 Comprender (K2)

Siete actividades — a menudo iterativas, no estrictamente secuenciales:

1. **Planificación del testing** — Definir objetivos y enfoque
2. **Monitoreo y control del testing** — Hacer seguimiento del progreso; tomar acciones correctivas
3. **Análisis del testing** — ¿*Qué* probar? Identificar condiciones de prueba a partir de la base de pruebas
4. **Diseño del testing** — ¿*Cómo* probar? Convertir condiciones en casos de prueba y definir datos de prueba
5. **Implementación del testing** — Preparar scripts de prueba, suites, configuración del entorno
6. **Ejecución del testing** — Ejecutar pruebas, registrar resultados, reportar anomalías
7. **Cierre del testing** — Archivar el testware, escribir el informe de cierre, capturar lecciones aprendidas

El Proceso de Testing en Contexto — 💡 Comprender (K2)

Los factores de contexto influyen en la estrategia, técnicas, nivel de automatización, cobertura, detalle del testware y reportes:

Factor	Qué incluye
Interesados	Necesidades, expectativas, requisitos, disposición a cooperar
Miembros del equipo	Habilidades, conocimiento, disponibilidad, experiencia
Dominio de negocio	Criticidad, tolerancia al riesgo, mercado, requisitos legales/normativos
Factores técnicos	Tipo de tecnología, arquitectura del sistema, soporte de herramientas
Restricciones del proyecto	Alcance, tiempo, presupuesto, recursos
Factores organizacionales	Estructura organizacional, políticas existentes, cultura
Ciclo de vida del desarrollo de software	Modelo secuencial, iterativo, Agile o DevOps utilizado
Herramientas	Disponibilidad, usabilidad, licencias, cumplimiento

Consejo de examen (K2): Una pregunta de examen puede describir un escenario de proyecto y preguntar cómo un factor de contexto específico debería influir en el enfoque de testing. Conoce cada factor y qué afecta.

Testware — lo que produce cada actividad — 💡 Comprender (K2)

Actividad	Ejemplos de resultados
Planificación	Plan de pruebas, cronograma de pruebas, criterios de entrada, criterios de salida, registro de riesgos
Monitoreo y control	Informes de progreso
Análisis	Condiciones de prueba, informes de defectos (respecto a defectos en la base de pruebas)
Diseño	Casos de prueba, ítems de cobertura, requisitos del entorno de prueba
Implementación	Procedimientos de prueba, scripts/suites de prueba, datos de prueba, cronograma de ejecución de pruebas
Ejecución	Registros de prueba, informes de defectos
Cierre	Informe de cierre del testing, elementos de acción para la mejora, solicitudes de cambio, lecciones aprendidas documentadas

Trazabilidad — 💡 Comprender (K2)

Una buena trazabilidad vincula los casos de prueba a los requisitos y riesgos. Ayuda a: - Medir la cobertura - Analizar el impacto de los cambios - Apoyar auditorías e informes de gobernanza

Roles en el Testing — 💡 Comprender (K2)

- **Rol de gestión del testing** — Planificación, monitoreo, control y cierre; liderazgo del equipo
- **Rol de testing** — Análisis, diseño, implementación y ejecución

Habilidades Esenciales y Enfoques de Equipo — 💡 Comprender (K2)

Los buenos testers combinan: conocimiento del testing, minuciosidad, pensamiento analítico/crítico, creatividad, habilidades de comunicación y conocimiento técnico.

Enfoque de Equipo Completo — todos son responsables de la calidad; los testers colaboran estrechamente con los desarrolladores y representantes del negocio (proveniente de XP).

Excepción: los sistemas de seguridad crítica pueden requerir mayor independencia del testing.

Independencia del Testing — los testers independientes encuentran diferentes defectos debido a distintos sesgos cognitivos: 1. El autor prueba su propio trabajo (menor independencia) 2. Compañeros del mismo equipo 3. Testers de fuera del equipo (alta) 4. Testers de fuera de la organización (muy alta)

- **Beneficio:** Una perspectiva diferente detecta más defectos
- **Desventaja:** Riesgo de aislamiento, problemas de comunicación, que los desarrolladores pierdan la responsabilidad sobre la calidad, mentalidad de "nosotros vs. ellos"

Glosario Rápido de Términos Clave — 🧠 Recordar (K1)

Término	Definición
Error	Un error humano
Defecto	Una falla introducida por un error
Fallo	Comportamiento incorrecto causado por la ejecución de un defecto
Causa raíz	La razón fundamental por la que existe un defecto
Base de pruebas	Todo lo utilizado para derivar casos de prueba (requisitos, diseños, código, etc.)
Objeto de prueba	El ítem que está siendo probado
Testware	Todos los artefactos producidos por las actividades de testing
Verificación	¿Estamos construyendo el producto correctamente?
Validación	¿Estamos construyendo el producto correcto?

Preguntas de Práctica

- ¿Cuál es la diferencia entre error, defecto y fallo? Da un ejemplo de cada uno.
- Lista los 7 principios del testing y explica cualquiera de ellos con tus propias palabras.
- ¿Cuál es la diferencia entre testing y QA?
- ¿Cuál es la diferencia entre verificación y validación?

- ¿Por qué el testing exhaustivo es imposible?
- ¿Cuáles son los dos roles principales en el testing y en qué se enfoca cada uno?
- ¿Cuáles son los beneficios y desventajas de la independencia del testing?
- ¿Con qué ayuda la trazabilidad en el proceso de testing?

Capítulo 2 - El Testing a lo Largo del Ciclo de Vida del Desarrollo de Software

La idea central: El testing no ocurre al final. Vive dentro de cada fase del desarrollo, se adapta al modelo que utilice el equipo y cambia de forma según el contexto.

Objetivos de Aprendizaje del Capítulo 2

LO	K	Tema
FL-2.1.1	K2	Explicar el impacto del modelo SDLC elegido sobre el testing
FL-2.1.2	K1	Recordar buenas prácticas de testing aplicables a cualquier modelo SDLC
FL-2.1.3	K1	Recordar ejemplos de TDD, ATDD y BDD
FL-2.1.4	K2	Resumir los beneficios de DevOps para el testing
FL-2.1.5	K2	Explicar el enfoque shift-left
FL-2.1.6	K2	Explicar cómo las retrospectivas pueden usarse para la mejora del proceso en el testing
FL-2.2.1	K2	Distinguir los diferentes niveles de prueba
FL-2.2.2	K2	Distinguir los diferentes tipos de prueba
FL-2.2.3	K2	Distinguir el testing de confirmación del testing de regresión
FL-2.3.1	K2	Resumir los disparadores del testing de mantenimiento
FL-2.3.2	K2	Explicar el rol del análisis de impacto en el testing de mantenimiento

Consejo de examen: No hay K3 en este capítulo. Los ítems K1 son FL-2.1.2 y FL-2.1.3 — solo recordar, no se requiere explicación. Todo lo demás requiere que *expliques* o *distingas*.

2.1 El Testing en el Contexto de un SDLC — 💡 Comprender (K2)

El modelo SDLC elegido impacta directamente **cómo** se realiza el testing — no si se realiza.

Modelo	Ejemplos	Cómo encaja el testing
Secuencial	Waterfall, modelo V	El testing comienza después del desarrollo; el modelo V mapea cada etapa de desarrollo a un nivel de prueba
Iterativo	Espiral, Prototipado	El testing ocurre dentro de cada ciclo repetido
Incremental	Proceso Unificado	Cada pequeño incremento es probado antes de pasar al siguiente

En la práctica, muchos proyectos reales combinan modelos — p. ej., Agile usa enfoques tanto iterativos como incrementales. Las prácticas Agile como Scrum, Kanban, TDD, BDD y XP son métodos que operan *dentro* de estos tipos de modelos.

El SDLC impacta el testing en 5 maneras: - Alcance y temporización de las actividades de testing - Nivel de detalle de la documentación de pruebas - Elección de técnicas y enfoque de testing - Grado de automatización del testing - Roles y responsabilidades de los testers

Buenas prácticas de testing — aplican a CUALQUIER modelo SDLC — 🧠 Recordar (K1)

- Cada actividad de desarrollo tiene una actividad de testing correspondiente
- Los diferentes niveles de prueba tienen objetivos de testing específicos y distintos
- El análisis y diseño del testing comienzan durante la fase de desarrollo correspondiente
- Los testers revisan los productos de trabajo tan pronto como los borradores están disponibles

El Testing como Motor: TDD, ATDD y BDD — 🧠 Recordar (K1)

Las pruebas se escriben **antes** del código en los tres enfoques. Los tres implementan el principio de testing temprano, apoyan el shift left y soportan un modelo de desarrollo iterativo.

Enfoque	Quién lo usa	Cómo funciona
TDD (Desarrollo Guiado por Pruebas)	Desarrolladores	Escribe una prueba que falla → escribe el código para pasarla → refactoriza. TDD fue diseñado para reemplazar el extenso diseño de software previo.
ATDD (Desarrollo Guiado por Pruebas de Aceptación)	Equipo completo	Las pruebas se derivan de los criterios de aceptación "como parte del proceso de diseño del sistema" antes de que se construya la funcionalidad.
BDD (Desarrollo Guiado por Comportamiento)	Equipo completo	Las pruebas se escriben en lenguaje natural usando el formato Dado/Cuando/Entonces; los casos de prueba "deben traducirse automáticamente a pruebas ejecutables."

Consejo de examen (K1): Para FL-2.1.3, solo necesitas recordar quién usa cada enfoque y su idea central. Los detalles de "cómo funciona" en la tabla son de contexto — el examen evalúa reconocimiento, no reproducción del procedimiento.

DevOps y el Testing — 💡 Comprender (K2)

DevOps une desarrollo, testing y operaciones para entregar software de alta calidad más rápido.

Beneficios para el testing: - Retroalimentación rápida sobre la calidad del código mediante pipelines CI/CD - Pruebas de regresión automatizadas que se ejecutan en cada commit - Mayor visibilidad sobre la calidad no funcional (rendimiento, confiabilidad) - Reduce el testing manual repetitivo mediante la automatización - CI promueve el shift left al incentivar a los desarrolladores a enviar código de alta calidad con pruebas de componentes y análisis estático - CI/CD automatizado facilita el establecimiento de entornos de prueba estables

Incluso con una gran automatización, el testing manual — especialmente desde la perspectiva del usuario — sigue siendo necesario.

Shift Left — 💡 Comprender (K2)

Shift left significa **probar más temprano en el SDLC** — no esperar a que el código esté completo.

Buenas prácticas para lograr el shift left: - Revisar las especificaciones desde la perspectiva del tester (encontrar ambigüedades a tiempo) - Escribir casos de prueba antes de escribir el código - Usar CI/CD para obtener retroalimentación rápida en cada commit de código - Realizar análisis estático antes del testing dinámico - Comenzar el testing no funcional a nivel de componente, donde sea posible

El shift left puede aumentar el esfuerzo y el costo temprano, pero ahorra significativamente más adelante.

Importante: Los interesados deben ser convencidos y comprometidos con el concepto. Sin el compromiso de los interesados, el esfuerzo adicional inicial no puede justificarse.

Retrospectivas y Mejora del Proceso — 💡 Comprender (K2)

Se realizan al final de una iteración, proyecto o hito de lanzamiento. Los participantes incluyen testers, desarrolladores, arquitectos, product owners y analistas de negocio.

Beneficios para el testing: - Mayor efectividad y eficiencia del testing - Mejor calidad del testware - Cohesión y aprendizaje del equipo - Mejor cooperación entre desarrollo y testing - Mejor calidad de la base de pruebas — se identifican y abordan las deficiencias en el alcance/calidad de los requisitos

2.2 Niveles y Tipos de Prueba — 💡 Comprender (K2)

Niveles de Prueba — los cinco niveles — 💡 Comprender (K2)

Nivel	Enfoque	Típicamente realizado por
Testing de componentes (unitario)	Componentes individuales en aislamiento	Desarrolladores
Testing de integración de componentes	Interfaces e interacciones entre componentes	Desarrolladores / Testers
Testing de sistema	Comportamiento de extremo a extremo del sistema completo	Testers independientes
Testing de integración de sistema	Interfaces entre el sistema y sistemas/servicios externos	Testers
Testing de aceptación	Validación; demostrar la preparación para el despliegue	Negocio / Usuarios finales

Cómo se distinguen los niveles de prueba (K2): Los niveles de prueba difieren entre sí por: **objeto de prueba, objetivos del testing, base de pruebas, defectos y fallos** (tipos típicamente encontrados), y **enfoque y responsabilidades**.

Subtipos del Testing de Aceptación: - **UAT:** Los usuarios reales validan que el sistema cumple con sus necesidades - **OAT:** Preparación operacional — respaldo, recuperación, seguridad -

Contractual/Regulatorio: Verifica el cumplimiento con contratos o regulaciones - **Testing alfa:** Usuarios internos en las instalaciones del desarrollador - **Testing beta:** Usuarios externos en su propio entorno

Tipos de Prueba — qué aspecto se está probando — 💡 Comprender (K2)

Tipo	Qué verifica
Testing funcional	Lo que el sistema hace — completitud funcional, corrección, adecuación
Testing no funcional	Qué tan bien se comporta el sistema — características ISO/IEC 25010: Adecuación funcional, Eficiencia de rendimiento, Compatibilidad, Usabilidad, Fiabilidad, Seguridad, Mantenibilidad, Portabilidad, Seguridad funcional
Testing de caja negra	Comportamiento basado en especificaciones, sin conocimiento de la estructura interna
Testing de caja blanca	Estructura interna — rutas de código, arquitectura, flujos de datos

Los cuatro tipos de prueba pueden aplicarse en **cualquier** nivel de prueba.

Nota: "Caja negra" y "caja blanca" aparecen tanto en el Capítulo 2 (como *tipos* de prueba) como en el Capítulo 4 (como categorías de *técnicas* de prueba). En el Capítulo 2 clasifican *en qué se basan las pruebas* (especificación vs. estructura); en el Capítulo 4 clasifican *cómo se derivan los casos de prueba*. Mismos términos, significado relacionado pero distinto — no te dejes confundir.

Testing Relacionado con Cambios — 💡 Comprender (K2)

- **Testing de confirmación:** Verifica que un defecto específico ha sido corregido
- Enfoque 1: Ejecutar todos los casos de prueba que previamente fallaron debido al defecto
- Enfoque 2: Agregar nuevas pruebas para cubrir los cambios realizados para corregir el defecto
- Cuando el tiempo es escaso: puede limitarse a reproducir el fallo que generó el informe del defecto
- **Testing de regresión:** Asegura que los cambios no hayan roto la funcionalidad existente

El testing de regresión crece con cada versión — es un fuerte candidato para la automatización.

2.3 Testing de Mantenimiento — 💡 Comprender (K2)

Se activa cuando un sistema que ya está en producción necesita cambiar.

Disparadores (categorías v4) — 💡 Comprender (K2)

Disparador	Ejemplos
Modificaciones	Mejoras planificadas, cambios correctivos (corrección de bugs), hot fixes
Actualizaciones o migraciones	Migración a una nueva plataforma, sistema operativo o entorno
Retiro	Fin de vida — testing de archivado y recuperación de datos

El alcance depende de: - Grado de riesgo del cambio - Tamaño del sistema existente - Tamaño del cambio

Análisis de Impacto — 💡 Comprender (K2)

- Determina qué partes del sistema podrían verse afectadas por un cambio
- Define el alcance del testing de regresión
- Depende de una buena trazabilidad

Término	Definición
SDLC	Ciclo de Vida del Desarrollo de Software — el proceso para planificar, crear, probar y entregar software
Nivel de prueba	Un grupo de actividades de testing organizadas en conjunto (componente, integración, sistema, aceptación)
Tipo de prueba	Clasificación por lo que está siendo probado (funcional, no funcional, caja negra, caja blanca)
Shift left	Probar más temprano en el ciclo de vida para detectar defectos antes
TDD	Las pruebas se escriben antes del código para guiar el desarrollo
BDD	Las pruebas se escriben en formato de lenguaje natural Dado/Cuando/Entonces
Testing de regresión	Re-ejecución de pruebas para detectar efectos secundarios no deseados de un cambio
Testing de confirmación	Re-ejecución de pruebas después de una corrección para verificar que el defecto fue resuelto
Análisis de impacto	Evaluar qué partes del sistema se ven afectadas por un cambio

Preguntas de Práctica

- ¿Cuáles son las tres categorías principales de modelos SDLC? Da un ejemplo de cada una.
- ¿Cómo impacta la elección del modelo SDLC en el testing? Nombra al menos tres formas.
- ¿Cuál es la diferencia entre TDD, ATDD y BDD?
- ¿Cuáles son los beneficios y riesgos de DevOps para el testing?
- ¿Qué significa "shift left" y cómo puedes lograrlo?
- ¿Cuál es la diferencia entre el testing de componentes y el testing de integración de componentes?
- ¿Cuál es la diferencia entre el testing de confirmación y el testing de regresión?
- ¿Cuáles son los tres disparadores del testing de mantenimiento en v4?
- ¿Por qué el testing de regresión es un buen candidato para la automatización?
- ¿Cuál es el propósito del análisis de impacto en el testing de mantenimiento?

Capítulo 3 - Pruebas Estáticas

La idea principal: No es necesario ejecutar el software para encontrar problemas en él. Revisar documentos, diseños y código antes de la ejecución frecuentemente detecta defectos de forma más temprana, más económica y en lugares que las pruebas dinámicas simplemente no pueden alcanzar.

Niveles Cognitivos en Este Capítulo

K1 = Recordar · K2 = Comprender · K3 = Aplicar — consultar el Capítulo 1 para la leyenda completa.

Objetivos de Aprendizaje del Capítulo 3

LO	K	Tema
FL-3.1.1	K1	Reconocer los tipos de productos de trabajo que pueden examinarse mediante pruebas estáticas
FL-3.1.2	K2	Explicar el valor de las pruebas estáticas
FL-3.1.3	K2	Explicar la diferencia entre pruebas estáticas y pruebas dinámicas
FL-3.2.1	K1	Identificar los beneficios de la retroalimentación temprana y frecuente de las partes interesadas
FL-3.2.2	K2	Resumir las actividades del proceso de revisión
FL-3.2.3	K1	Recordar qué responsabilidades se asignan a los roles principales al realizar revisiones
FL-3.2.4	K2	Distinguir entre los diferentes tipos de revisión
FL-3.2.5	K1	Recordar los factores de éxito de las revisiones

Consejo para el examen: No hay K3 en este capítulo. Cuatro ítems K1 (FL-3.1.1, FL-3.2.1, FL-3.2.3, FL-3.2.5) — pura memorización. FL-3.1.3 y FL-3.2.4 requieren *explicar diferencias* y *distinguir* — K2 clásico.

3.1 Fundamentos de las Pruebas Estáticas — Comprender (K2)

- Evalúa un producto de trabajo **sin ejecutarlo**
- Se realiza **manualmente** (revisiones) o **con herramientas** (análisis estático — linters, analizadores de código en CI)
- Aplica tanto a verificación como a validación
- Objetivos: mejorar la calidad, detectar defectos, evaluar legibilidad, completitud, corrección, testabilidad y consistencia

¿Qué Puede Examinarse con Pruebas Estáticas? — 🧠 Recordar (K1)

- Casi **cualquier producto de trabajo** que un humano pueda leer y comprender: requisitos, código fuente, planes/casos de prueba, ítems del backlog, chárters de prueba, documentos del proyecto, contratos, modelos
- El **análisis estático** requiere una sintaxis formal (código, modelos, texto estructurado)
- **NO es apropiado para:** productos de trabajo que los humanos no pueden interpretar o que no pueden ser analizados con herramientas por razones legales (p. ej., ejecutables de terceros)

¿Por Qué Hacerlo? El Valor de las Pruebas Estáticas — 💡 Comprender (K2)

- Detecta defectos **de forma temprana** — antes de que se propaguen y se vuelvan costosos
- Encuentra cosas que **las pruebas dinámicas no pueden** — código inalcanzable, fallas de diseño, defectos en documentos no ejecutables
- Genera **comprensión compartida** entre el equipo
- Es **rentable** — el costo de una revisión anticipada << el costo de un defecto tardío en el proyecto

Defectos más fáciles de encontrar mediante pruebas estáticas — 💡 Comprender (K2)

Categoría	Ejemplos
Defectos en requisitos	Inconsistencias, ambigüedades, contradicciones, omisiones, duplicaciones
Defectos de diseño	Estructuras de base de datos ineficientes, modularización deficiente
Defectos de codificación	Valores no definidos, variables no declaradas, código inalcanzable/duplicado, complejidad excesiva
Desviaciones de estándares	Violaciones de convenciones de nomenclatura, violaciones de estándares de codificación
Defectos de interfaz	Número, tipo u orden de parámetros que no coinciden
Vulnerabilidades de seguridad	Desbordamientos de búfer
Brechas en la base de pruebas	Pruebas faltantes para un criterio de aceptación

Pruebas Estáticas vs. Pruebas Dinámicas — 💡 Comprender (K2)

Aspecto	Pruebas Estáticas	Pruebas Dinámicas
¿Requiere ejecución?	No	Sí
Cómo se encuentran los defectos	Directamente	A través de fallas → análisis de causa raíz
Aplicable a	Productos de trabajo ejecutables Y no ejecutables	Solo productos de trabajo ejecutables
Caminos de código poco frecuentes	Más fácil de detectar	Difícil de alcanzar
Características de calidad	Mantenibilidad, legibilidad	Rendimiento, fiabilidad

Piénsalo así: **las pruebas estáticas leen la receta**, mientras que **las pruebas dinámicas prueban el plato**.

3.2 Retroalimentación y Proceso de Revisión — 💡 Comprender (K2)

Por Qué Importa la Retroalimentación Temprana y Frecuente de las Partes Interesadas — 🧠 Recordar (K1)

- Sin la participación regular de las partes interesadas, el producto se aleja de la visión original
- Consecuencias: retrabajo costoso, plazos incumplidos, culpa asignada, fracaso del proyecto
- Retroalimentación frecuente: previene malentendidos de forma temprana, mantiene al equipo enfocado en el valor, aborda riesgos de forma proactiva

El Proceso de Revisión — 5 Actividades — 🧠 Recordar (K1)

Basado en ISO/IEC 20246:

Paso	Qué ocurre
1. Planificación	Definir alcance, propósito, criterios de salida, esfuerzo y plazos
2. Inicio de la revisión	Asegurarse de que todos los participantes tengan acceso, entiendan su rol y estén preparados
3. Revisión individual	Cada revisor evalúa el producto de trabajo de forma independiente; registra anomalías, recomendaciones y preguntas
4. Comunicación y análisis	Las anomalías se discuten en una reunión de revisión; se toman decisiones sobre el estado, la propiedad y las acciones de seguimiento
5. Corrección e informe	El autor corrige los defectos encontrados; el líder de revisión redacta el informe de revisión; el producto de trabajo se acepta una vez que se cumplen los criterios de salida

Nota práctica: El tamaño de muchos productos de trabajo los hace demasiado grandes para cubrirse en una sola revisión. El proceso de revisión puede invocarse múltiples veces para completar la revisión de todo el producto de trabajo.

Roles en una Revisión — 🧠 Recordar (K1)

Rol	Responsabilidad
Gerente	Decide qué se revisa; proporciona recursos (personas, tiempo)
Autor	Crea y posteriormente corrige el producto de trabajo bajo revisión
Moderador (facilitador)	Dirige la reunión de revisión; gestiona la mediación, el tiempo y garantiza un entorno seguro
Secretario (registrador)	Registra anomalías, decisiones y nuevos hallazgos durante la reunión
Revisor	Realiza la revisión real; puede ser un miembro del equipo, un experto en la materia o cualquier parte interesada
Líder de revisión	Es responsable de toda la revisión; decide quién participa, cuándo y dónde

Truco de memorización: Piensa en una revisión como una reunión estructurada con un **jefe** (gerente), un **creador** (autor), un **árbitro** (moderador), un **tomador de notas** (secretario), **evaluadores** (revisores) y un **organizador** (líder de revisión).

Tipo	Formalidad	Dirigida por	Características clave
Revisión informal	La más baja	Cualquier persona	Sin proceso definido; sin resultados formales; solo encontrar anomalías
Walkthrough	Baja–Media	Autor	El autor guía al equipo a través del producto de trabajo; objetivos: evaluar la calidad y generar confianza en el producto de trabajo, educar a los revisores, lograr consenso, generar nuevas ideas, motivar y permitir que los autores mejoren, detectar anomalías; Secretario: No requerido (opcional — el autor también puede asumir este rol); Los revisores podrían realizar una revisión individual antes del walkthrough, pero no es requerido
Revisión técnica	Media–Alta	Moderador	Revisores técnicamente calificados; objetivos: alcanzar consenso y tomar decisiones sobre problemas técnicos, detectar anomalías, evaluar la calidad, generar confianza en el producto de trabajo, generar nuevas ideas, motivar a los autores a mejorar
Inspección	La más alta	Moderador (NO el autor)	Sigue el proceso completo definido; la detección máxima de anomalías es el objetivo principal; se recopilan métricas; el autor no puede ser líder de revisión ni secretario

Punto clave del examen: En una inspección, el **autor no puede actuar como líder de revisión ni secretario** — esto es exclusivo de este tipo de revisión.

Consejo para el examen: El programa NO exige un secretario para los walkthroughs. La revisión individual previa también es "no requerida".

Factores que afectan la formalidad requerida: El nivel requerido de formalidad de revisión depende de: el SDLC que se siga, la madurez del proceso de desarrollo, la criticidad y complejidad del producto de trabajo, los requisitos legales o regulatorios, y la necesidad de un rastro de auditoría.

Factores de Éxito para las Revisiones — Recordar (K1)

- Los objetivos son **claros y medibles** — la evaluación de los participantes **nunca** es un objetivo
- Se elige el **tipo de revisión correcto** para el producto de trabajo y el contexto
- Las revisiones se realizan en **fragmentos pequeños** para que los revisores se mantengan enfocados
- Se da **retroalimentación** a las partes interesadas y a los autores para que puedan mejorar
- Los participantes tienen **tiempo de preparación adecuado**
- La **dirección apoya** el proceso de revisión
- Las revisiones forman parte de la **cultura de la organización**
- Todos los participantes reciben **capacitación adecuada**
- Las reuniones se **facilitan de manera efectiva**

Término	Definición
Pruebas estáticas	Evaluar un producto de trabajo sin ejecutarlo
Pruebas dinámicas	Probar mediante la ejecución del software
Análisis estático	Evaluación asistida por herramientas contra una sintaxis formal
Revisión	Evaluación manual de un producto de trabajo por una o más personas
Inspección	Tipo de revisión más formal; sigue un proceso completo definido; el autor no puede ser líder ni secretario
Revisión técnica	Revisión dirigida por un moderador con revisores técnicamente calificados
Walkthrough	Revisión dirigida por el autor
Revisión informal	Revisión sin proceso definido ni resultados formales
Anomalía	Cualquier resultado que difiere de lo esperado; identificado durante una revisión
Moderador	Garantiza el funcionamiento efectivo de una reunión de revisión

Preguntas de Práctica

- ¿Qué son las pruebas estáticas y en qué se diferencian de las pruebas dinámicas?
- ¿Qué tipos de productos de trabajo pueden examinarse mediante pruebas estáticas?
- Nombra tres tipos de defectos que son más fáciles de encontrar mediante pruebas estáticas.
- ¿Cuáles son las cinco actividades del proceso de revisión?
- ¿Cuáles son los seis roles principales en una revisión? ¿Qué hace cada uno?
- ¿Cuál es el tipo de revisión más formal y qué lo hace único?
- ¿Cuál es la diferencia principal entre un walkthrough y una revisión técnica?
- ¿Por qué el autor no puede ser líder de revisión ni secretario en una inspección?
- Lista cuatro factores de éxito para las revisiones.
- ¿Por qué la evaluación de los participantes nunca debe ser un objetivo de una revisión?

Capítulo 4 - Análisis y Diseño de Pruebas

La idea principal: ¿Cómo decides *qué* probar y *cómo* probarlo sin simplemente adivinar? Las técnicas de prueba te brindan una forma sistemática y repetible de diseñar casos de prueba que cubren lo que importa — basándose en especificaciones, estructura del código o tu propia experiencia.

Niveles Cognitivos en Este Capítulo

K1 = Recordar · K2 = Comprender · K3 = Aplicar — consultar el Capítulo 1 para la leyenda completa.

Objetivos de Aprendizaje del Capítulo 4

LO	K	Tema
FL-4.1.1	K2	Resumir las categorías de técnicas de prueba y sus subcategorías
FL-4.2.1	K3	Aplicar la partición de equivalencia para derivar casos de prueba para un escenario dado
FL-4.2.2	K3	Aplicar el análisis de valores límite para derivar casos de prueba para un escenario dado
FL-4.2.3	K3	Aplicar las pruebas de tabla de decisión para derivar casos de prueba para un escenario dado
FL-4.2.4	K3	Aplicar las pruebas de transición de estado para derivar casos de prueba para un escenario dado
FL-4.3.1	K2	Explicar las pruebas de sentencias
FL-4.3.2	K2	Explicar las pruebas de ramas
FL-4.3.3	K2	Explicar el valor de las pruebas de caja blanca
FL-4.4.1	K2	Explicar la adivinación de errores
FL-4.4.2	K2	Explicar las pruebas exploratorias
FL-4.4.3	K2	Explicar las pruebas basadas en listas de verificación
FL-4.5.1	K2	Explicar los factores que conforman una buena historia de usuario (criterios INVEST)
FL-4.5.2	K2	Recordar los formatos típicos para redactar criterios de aceptación
FL-4.5.3	K3	Usar ATDD para derivar casos de prueba

Consejo para el examen: Las cuatro técnicas de caja negra (FL-4.2.1–4.2.4) son **K3** — el examen te dará un escenario y te pedirá que produzcas casos de prueba. Practica derivarlos, no solo explicarlos. Las técnicas de caja blanca (FL-4.3.1–4.3.2) son solo K2 — explicas la cobertura de sentencias y ramas, no la calculas.

4.1 Las Tres Categorías de Técnicas de Prueba — 💡 Comprender (K2)

Categoría	También conocida como	Basada en	Característica clave
Caja negra	Basada en especificaciones	Comportamiento especificado	Las pruebas son independientes de la implementación — sobreviven a los cambios de código
Caja blanca	Basada en estructura	Estructura interna/código	Las pruebas dependen de cómo está construido el software; se crean tras el diseño/implementación
Basada en experiencia	—	Conocimiento y habilidad del tester	Complementaria a las otras dos; encuentra defectos que las otras pasan por alto

Truco de memorización: Negro = no puedes ver dentro de la caja. Blanco = puedes ver todo dentro. Experiencia = ya has visto cajas similares romperse antes.

4.2 Técnicas de Prueba de Caja Negra — ⚙️ Aplicar (K3)

4.2.1 Partición de Equivalencia (EP) — ⚙️ Aplicar (K3)

- Divide los datos de entrada/salida en **particiones** donde todos los valores se procesan de manera idéntica
- **Un caso de prueba por partición** es suficiente
- **Partición válida** = valores que el sistema acepta; **Partición inválida** = valores que el sistema rechaza
- **Cobertura de EP al 100%** = cada partición (válida E inválida) ejercida al menos una vez
- Cobertura = particiones ejercidas ÷ total de particiones
- **Múltiples entradas:** usar **cobertura Each Choice** — al menos un valor de cada conjunto de particiones en al menos un caso de prueba

Tipos de partición: Las particiones pueden ser continuas o discretas, ordenadas o no ordenadas, finitas o infinitas. Las fuentes de particiones incluyen no solo los parámetros de entrada sino también: ítems de configuración, valores internos, valores relacionados con el tiempo y parámetros de interfaz. Las particiones deben ser no vacías y no solapadas.

Ejemplo: Un campo acepta edades de 18 a 65. Particiones: menores de 18 (inválida), 18–65 (válida), mayores de 65 (inválida). Necesitas al menos 3 casos de prueba — uno por partición.

4.2.2 Análisis de Valores Límite (BVA) — ⚙️ Aplicar (K3)

- Extiende EP apuntando a los **bordes** de las particiones ordenadas — donde los desarrolladores cometen más errores
- Solo aplica a particiones **ordenadas** (donde existen mínimos/máximos)

Dos versiones:

Versión	Ítems de cobertura por límite	¿Más rigurosa?
BVA de 2 valores	Valor límite + su vecino más cercano de la partición adyacente	Menos
BVA de 3 valores	El valor límite en sí, el valor inmediatamente por debajo y el valor inmediatamente por encima	Más — puede detectar errores por desplazamiento de uno que el de 2 valores no detecta

Ejemplo: Para un rango de 1–10 usando BVA de 3 valores, probar: 0, 1, 2 (límite inferior) y 9, 10, 11 (límite superior).

Cobertura al 100%: Ejercer todos los valores límite (2 valores) o todos los valores límite y sus vecinos (3 valores).

4.2.3 Pruebas de Tabla de Decisión — Aplicar (K3)

- Mapea cada **combinación de condiciones** a las acciones esperadas — ideal para reglas de negocio complejas
- Cada **columna** = una regla de decisión (combinación única de condiciones + acciones resultantes)
- Notación: **T/F** = condición verdadera/falsa; **–** = irrelevante; **N/A** = inviable; **X** = la acción ocurre; **en blanco** = la acción no ocurre
- **Cobertura al 100%** = todas las columnas viables ejercidas; Cobertura = columnas viables ejercidas ÷ total de columnas viables
- **Fortaleza:** revela brechas/contradicciones en los requisitos; nada se pasa por alto
- **Debilidad:** las reglas crecen exponencialmente (2^n) — usar una tabla minimizada o un enfoque basado en riesgos

Nota: La cobertura se mide contra las columnas reales de la tabla que se está usando. Si la tabla ha sido minimizada (reglas colapsadas), la cobertura se calcula contra sus columnas reales — no contra el máximo teórico de 2^n .

Tablas de decisión de entrada extendida: Los ejemplos anteriores muestran tablas de entrada limitada (las condiciones son booleanas: verdadero/falso). Las tablas de entrada extendida permiten que las condiciones tomen múltiples valores (rangos, particiones o valores discretos) y que las acciones especifiquen valores de salida exactos. Son más compactas pero más difíciles de leer.

4.2.4 Pruebas de Transición de Estado — Aplicar (K3)

- Prueba sistemas cuyo comportamiento depende del **estado actual**
- **Diagrama de estados:** muestra todos los estados y transiciones válidas (**evento** [condición de guarda] / acción)
- **Tabla de estados:** filas = estados, columnas = eventos; celdas vacías = transiciones inválidas
- Un caso de prueba = una secuencia de eventos que produce una secuencia de cambios de estado

Tres criterios de cobertura (del más débil al más fuerte):

Cobertura	Qué ejercita	Notas
Todos los estados	Todos los estados al menos una vez	El más débil — puede lograrse sin probar todas las transiciones
Transiciones válidas (0-switch)	Todas las transiciones válidas — cada transición válida ejercida al menos una vez, sin encadenar secuencias (0-switch = solo pasos individuales)	Criterio más ampliamente utilizado
Todas las transiciones	Todas las transiciones válidas + todas las inválidas	El más fuerte; requerido para software crítico para la misión/seguridad

Punto clave del examen: Lograr la cobertura de transiciones válidas garantiza la cobertura de todos los estados. Lograr la cobertura de todas las transiciones garantiza ambas. Probar solo **una** transición inválida por caso de prueba para evitar el enmascaramiento de defectos.

4.3 Técnicas de Prueba de Caja Blanca — 💡 Comprender (K2)

4.3.1 Pruebas de Sentencias — 💡 Comprender (K2)

- Ítems de cobertura: **sentencias ejecutables**
- **Cobertura de sentencias al 100%** = cada sentencia ejecutable ejecutada al menos una vez
- Cobertura = sentencias ejercidas ÷ total de sentencias ejecutables
- **Limitación:** NO garantiza que todas las ramas sean probadas — los defectos específicos de una condición pueden pasarse por alto

4.3.2 Pruebas de Ramas — 💡 Comprender (K2)

- Ítems de cobertura: **ramas** (transferencias de control entre nodos en un grafo de flujo de control)
- Ramas: **incondicionales** (línea recta) o **condicionales** (if/else, switch, bucles)
- **Cobertura de ramas al 100%** = todas las ramas ejercidas; Cobertura = ramas ejercidas ÷ total de ramas

Regla crítica: La cobertura de ramas subsume la cobertura de sentencias. Si logras el 100% de cobertura de ramas, automáticamente logras el 100% de cobertura de sentencias. Lo contrario NO es verdad.

Limitación: No detecta defectos que solo aparecen en un camino específico a través del código.

4.3.3 El Valor de las Pruebas de Caja Blanca — 💡 Comprender (K2)

- **Fortaleza:** toma en cuenta toda la implementación — encuentra defectos incluso cuando la especificación es vaga o incompleta
- **Debilidad:** si una función requerida está **completamente ausente** (defecto de omisión), la caja blanca no lo detectará
- Utilizable tanto en **pruebas estáticas** (análisis de código/pseudocódigo) como en **pruebas dinámicas** (ejecución del código)

Las pruebas de caja negra por sí solas no dan ninguna medida de la cobertura real del código. Las pruebas de caja blanca proporcionan esa medición objetiva.

4.4 Técnicas de Prueba Basadas en Experiencia — 💡 Comprender (K2)

4.4.1 Adivinación de Errores — 💡 Comprender (K2)

- Utiliza el conocimiento del tester sobre fallas pasadas, errores típicos de los desarrolladores y tipos comunes de fallas
- **Ataques de fallas:** lista estructurada de errores/defectos probables → diseñar pruebas específicamente para exponerlos
- Las listas provienen de la experiencia, datos de defectos o conocimiento común de fallas

4.4.2 Pruebas Exploratorias — 💡 Comprender (K2)

- Diseñar, ejecutar y evaluar pruebas **simultáneamente** mientras se aprende sobre el sistema
- **Basadas en sesiones:** sesiones de tiempo limitado guiadas por un **chárter de prueba** (contiene objetivos de prueba); terminan con un **debriefing**
- **Más útil cuando:** hay pocas especificaciones o ninguna, hay presión de tiempo, o para complementar técnicas formales
- Más efectivo con testers experimentados que tienen conocimiento del dominio, curiosidad y pensamiento analítico

4.4.3 Pruebas Basadas en Listas de Verificación — 💡 Comprender (K2)

- Utiliza una lista preparada de condiciones de prueba (frecuentemente formuladas como preguntas) para guiar las pruebas
- Construida a partir de la experiencia, el conocimiento del usuario y la comprensión de cómo falla el software
- Debe **actualizarse regularmente** basándose en el análisis de defectos — las listas de verificación pueden quedar obsoletas
- **Excluir:** ítems verificables automáticamente, ítems de criterios de entrada/salida, ítems demasiado generales
- **Compensación:** cobertura amplia y consistencia, pero menos repetibilidad que los casos de prueba detallados

4.5 Enfoques de Prueba Basados en Colaboración — 💡 Comprender (K2)

Se enfocan en **evitar defectos** a través de la comprensión compartida, no solo en encontrarlos.

4.5.1 Escritura Colaborativa de Historias de Usuario — 💡 Comprender (K2)

- Una **historia de usuario** representa una funcionalidad valiosa para un usuario o comprador
- **Las 3 C:** Tarjeta (el medio), Conversación (cómo se usará), Confirmación (criterios de aceptación)
- **Formato estándar:** *"Como [rol], quiero [objetivo], para poder [valor de negocio resultante]."*
- **Las buenas historias de usuario siguen INVEST:** Independent (Independiente), Negotiable (Negociable), Valuable (Valiosa), Estimable (Estimable), Small (Pequeña), Testable (Testable)

Técnicas de colaboración: Las historias de usuario se crean de forma colaborativa usando técnicas como **brainstorming** y **mapas mentales**. La colaboración reúne tres perspectivas: **negocio** (qué valor se necesita), **desarrollo** (qué es técnicamente factible) y **pruebas** (qué podría salir mal y qué necesita verificarse).

Si una parte interesada no puede imaginar cómo probar una historia de usuario, es una señal de que la historia no está clara o no es lo suficientemente valiosa.

4.5.2 Criterios de Aceptación — 💡 Comprender (K2)

- **Condiciones que una historia de usuario debe cumplir** para ser aceptada por las partes interesadas — estas son las condiciones de prueba de aceptación
- Se usan para: definir el alcance de la historia, alcanzar consenso, describir escenarios positivos/negativos, habilitar la planificación/estimación

Dos formatos más comunes:

Formato	Ejemplo
Orientado a escenarios (BDD)	Dado [contexto], Cuando [acción], Entonces [resultado]
Orientado a reglas	Lista de verificación en viñetas o tabla de mapeo entrada-salida

4.5.3 Desarrollo Guiado por Pruebas de Aceptación (ATDD) — ⚙️ Aplicar (K3)

- Escribir casos de prueba **antes** de implementar la historia de usuario, usando los criterios de aceptación como guía
- Pasos: (1) **Taller de especificación** — el equipo resuelve ambigüedades; (2) **Creación de casos de prueba** — basados en los criterios de aceptación, sirven como ejemplos del comportamiento correcto; (3) **Implementación** — los desarrolladores escriben código para pasar las pruebas
- Orden de pruebas: positivo (camino feliz) → negativo → características de calidad no funcionales
- Los casos de prueba deben estar en **lenguaje comprensible para las partes interesadas**

Regla de alcance de los casos de prueba: Los casos de prueba de ATDD deben cubrir todas las características de la historia de usuario y no deben ir más allá de la historia. Dos casos de prueba no deben describir las mismas características.

Cuando se capturan en formato de marco de automatización, las pruebas de aceptación se convierten en **requisitos ejecutables**.

Término	Definición
Técnica de caja negra	Deriva pruebas del comportamiento especificado; independiente de la estructura interna
Técnica de caja blanca	Deriva pruebas de la estructura interna/código
Técnica basada en experiencia	Usa el conocimiento y la experiencia del tester para diseñar pruebas
Partición de equivalencia	Divide los datos en particiones procesadas de forma idéntica; una prueba por partición
Análisis de valores límite	Prueba en los bordes de las particiones de equivalencia
Pruebas de tabla de decisión	Prueba combinaciones de condiciones y sus acciones resultantes
Pruebas de transición de estado	Prueba el comportamiento basándose en los estados del sistema y las transiciones entre ellos
Cobertura de sentencias	% de sentencias ejecutables ejercidas por las pruebas
Cobertura de ramas	% de ramas ejercidas; subsume la cobertura de sentencias
Adivinación de errores	Anticipar defectos basándose en la experiencia y el conocimiento de fallas
Pruebas exploratorias	Diseño, ejecución y evaluación simultáneos dentro de una sesión de tiempo limitado
Pruebas basadas en listas de verificación	Pruebas guiadas por una lista preparada de condiciones, frecuentemente como preguntas
Historia de usuario	Descripción de una funcionalidad valiosa para un usuario; tiene Tarjeta, Conversación, Confirmación
INVEST	Criterios para una buena historia de usuario: Independent, Negotiable, Valuable, Estimable, Small, Testable
Criterios de aceptación	Condiciones que una historia de usuario debe cumplir para ser aceptada; condiciones de prueba para las pruebas de aceptación
ATDD	Casos de prueba escritos antes de implementar la historia de usuario, basados en los criterios de aceptación

Preguntas de Práctica

- ¿Cuáles son las tres categorías de técnicas de prueba y en qué se diferencian?
- En la partición de equivalencia, ¿por qué se considera suficiente una prueba por partición?
- ¿Cuál es la diferencia entre BVA de 2 valores y BVA de 3 valores? ¿Cuándo usarías cada uno?
- ¿Qué significa cada símbolo en una tabla de decisión (T, F, –, N/A, X)?
- ¿Cuáles son los tres criterios de cobertura de transición de estado, del más débil al más fuerte?
- ¿Por qué debes probar solo una transición inválida por caso de prueba en las pruebas de transición de estado?

- ¿Qué significa que "la cobertura de ramas subsume la cobertura de sentencias"?
- ¿Cuál es la principal fortaleza y debilidad de las pruebas de caja blanca?
- ¿Qué es un ataque de fallas en la adivinación de errores?
- ¿Cuándo son más útiles las pruebas exploratorias?
- ¿Cuáles son las 3 C de una historia de usuario?
- ¿Qué significa INVEST?
- ¿Cuáles son los dos formatos más comunes para redactar criterios de aceptación?
- ¿Qué ocurre en el taller de especificación en ATDD?
- ¿Cuál es la diferencia principal entre las técnicas de detección de defectos (4.2–4.4) y los enfoques basados en colaboración (4.5)?

Capítulo 5 - Gestión de las Actividades de Prueba

La idea central: Las buenas pruebas no suceden solas — hay que planificarlas, hacerles seguimiento y cerrarlas correctamente. Este capítulo trata sobre el lado organizacional y de gestión de las pruebas: cómo definir la estrategia, estimar el esfuerzo, medir el progreso, gestionar el riesgo y reportar los resultados.

Niveles Cognitivos en Este Capítulo

K1 = Recordar · K2 = Comprender · K3 = Aplicar — ver Capítulo 1 para la leyenda completa.

Objetivos de Aprendizaje del Capítulo 5

LO	K	Tema
FL-5.1.1	K2	Ejemplificar el propósito y contenido de un plan de prueba
FL-5.1.2	K1	Reconocer cómo un tester aporta valor a la planificación de iteraciones y releases
FL-5.1.3	K2	Comparar los criterios de entrada y los criterios de salida
FL-5.1.4	K3	Aplicar técnicas de estimación para calcular el esfuerzo de prueba en un escenario definido
FL-5.1.5	K3	Aplicar la priorización de casos de prueba para un escenario dado
FL-5.1.6	K2	Recordar los conceptos de la pirámide de pruebas
FL-5.1.7	K2	Resumir los cuadrantes de prueba y su relación con los niveles y tipos de prueba
FL-5.2.1	K2	Explicar el propósito del monitoreo y control de pruebas y la diferencia entre ellos
FL-5.2.2	K2	Resumir las métricas utilizadas en las pruebas
FL-5.2.3	K2	Resumir el propósito y contenido de los informes de prueba
FL-5.3.1	K2	Explicar cómo los riesgos influyen en el alcance de las pruebas
FL-5.3.2	K1	Recordar la identificación de riesgos y la evaluación de riesgos
FL-5.4.1	K3	Aplicar un informe de defectos para un escenario dado
FL-5.4.2	K2	Resumir el contenido y propósito de un informe de defectos

Consejo para el examen: Hay tres ítems K3 aquí — estimación (FL-5.1.4), priorización (FL-5.1.5) e informes de defectos (FL-5.4.1). El examen te dará una situación y te pedirá producir algo: una estimación, una lista priorizada o un informe de defectos. FL-5.1.2 y FL-5.3.2 son K1 — puro recuerdo.

5.1 Planificación de Pruebas — 💡 Comprender (K2)

Un **plan de prueba** es un documento vivo que describe el alcance, el enfoque, el cronograma y los recursos para las pruebas.

Qué incluye normalmente un plan de prueba — 💡 Comprender (K2)

Sección	Contenido
Contexto	Alcance, base de prueba, restricciones, entorno de prueba
Objetivos	Lo que las pruebas buscan lograr
Partes interesadas	Roles, responsabilidades, comunicación
Enfoque de prueba	Estrategia, técnicas, tipos de prueba
Cronograma	Hitos, esfuerzo, plazos
Recursos	Personas, herramientas, entornos
Criterios de entrada y salida	Cuándo comenzar y cuándo detener las pruebas
Riesgos e incidencias	Riesgos conocidos y estrategias de mitigación
Métricas	Qué se medirá para hacer seguimiento del progreso

5.2 Contribución del Tester a la Planificación de Iteraciones y Releases — 🧠 Recordar (K1)

- **Planificación de releases:** estimar el esfuerzo a través de iteraciones, analizar riesgos, definir el enfoque de prueba
- **Planificación de iteraciones:** analizar historias de usuario, evaluar la testeabilidad, identificar criterios de aceptación, estimar por historia, participar en la identificación de riesgos

5.3 Criterios de Entrada y Criterios de Salida — 💡 Comprender (K2)

Criterios de entrada — cuándo INICIAR las pruebas — 💡 Comprender (K2)

- El testware está disponible (casos de prueba, scripts, datos)
- El entorno de prueba está listo y configurado
- El objeto de prueba ha superado una prueba de humo

Criterios de salida — cuándo DETENER las pruebas — 💡 Comprender (K2)

- Se han ejecutado las pruebas planificadas
- Se ha alcanzado el nivel de cobertura (sentencias, requisitos)
- La densidad de defectos está por debajo del umbral definido
- Se ha agotado el tiempo o el presupuesto (salida pragmática)

En Agile, los criterios de entrada/salida suelen denominarse **Definition of Ready (DoR)** y **Definition of Done (DoD)**.

5.4 Técnicas de Estimación — Aplicar (K3)

Técnicas de Estimación — Aplicar (K3)

Técnica	Cómo funciona
Estimación basada en ratios	Usa métricas de proyectos anteriores similares (p. ej., la relación entre el esfuerzo de prueba y el esfuerzo de desarrollo). Se aplica a los datos del proyecto actual para estimar el esfuerzo de prueba.
Extrapolación	Usa mediciones recopiladas al inicio del proyecto actual para estimar el esfuerzo restante. Cuantos más datos se recopilen, más precisa será la estimación.
Wideband Delphi	Técnica grupal basada en expertos. Los expertos estiman de forma independiente, comparten los resultados de forma anónima, discuten las diferencias y repiten hasta alcanzar el consenso. Planning Poker es una variante de Wideband Delphi adaptada para Agile.
Estimación de tres puntos	Produce tres estimaciones: la más optimista (a), la más probable (m), la más pesimista (b). Estimación final = $(a + 4m + b) / 6$. Reduce el sesgo y tiene en cuenta la incertidumbre.

Nota para el examen: Planning Poker es una **variante de Wideband Delphi**, no una técnica separada. Ambas están nombradas en el programa del curso.

5.5 Priorización de Casos de Prueba — Aplicar (K3)

La priorización determina el orden en que se ejecutan las pruebas.

Estrategias — Aplicar (K3)

Estrategia	Priorizar por
Basada en riesgos	Primero las áreas de mayor riesgo — mayor impacto si fallan
Basada en cobertura	Primero las pruebas que cubren más requisitos o más código
Basada en requisitos	Pruebas vinculadas a los requisitos más críticos o importantes

En la práctica, la priorización basada en riesgos es la más común y la más efectiva — maximiza la detección de defectos en el menor tiempo cuando el tiempo es limitado.

5.6 La Pirámide de Pruebas — 💡 Comprender (K2)

La pirámide de pruebas agrupa las pruebas por granularidad y recomienda una distribución entre niveles.

- **Base (unitaria):** muchas, rápidas, aisladas — baratas de escribir y mantener
- **Intermedia (integración):** prueba las interacciones entre componentes
- **Cima (UI/E2E):** pocas, lentas, frágiles — usar solo donde sea necesario

Importante: La pirámide de pruebas es un modelo, no una regla fija. **El número y los nombres de las capas pueden variar.** El principio: capas inferiores = más pruebas, más rápidas, más baratas; capas superiores = menos pruebas, más lentas, más costosas.

La pirámide implica: **automatizar al nivel más bajo posible.**

5.7 Cuadrantes de Prueba — 💡 Comprender (K2)

Dos ejes: **Orientado al negocio vs. Orientado a la tecnología** · **Apoyar al equipo vs. Criticar el producto**

	Apoyar al equipo	Criticar el producto
Orientado a la tecnología	Q1: Pruebas de componentes, pruebas de integración de componentes	Q4: Pruebas de rendimiento, pruebas de carga, pruebas de estrés, pruebas de seguridad, pruebas de humo (nota: las pruebas de usabilidad van en Q3, no en Q4)
Orientado al negocio	Q2: Pruebas funcionales, pruebas de historias, pruebas de historias de usuario, prototipos, simulaciones, pruebas de API	Q3: Pruebas exploratorias, pruebas de usabilidad, pruebas de aceptación de usuario

Los cuadrantes son una **herramienta de comunicación** — ayudan a los equipos a hablar sobre qué tipos de pruebas se necesitan y quién debe realizarlas. No son prescriptivos.

5.8 Riesgo y Pruebas — 💡 Comprender (K2)

Conceptos clave — 🧠 Recordar (K1)

- **Riesgo:** evento/peligro/amenaza potencial cuya ocurrencia causa un efecto adverso. Nivel de riesgo = probabilidad × impacto
- **Riesgo de producto:** riesgo de que algo en el sistema falle — determina el alcance de las pruebas
- **Riesgo de proyecto:** riesgo para el propio proyecto (cronograma, presupuesto, equipo) — gestionado a través de la gestión del proyecto

Categorías de Riesgo de Proyecto — 💡 Comprender (K2)

- **Organizacional:** recortes de financiación, prioridades en conflicto, cambios estructurales
- **Personas:** habilidades insuficientes, conflictos entre el personal, problemas de comunicación

- **Técnico:** fallos de tecnología, problemas con herramientas, problemas con el entorno
- **Proveedores:** fallos en la entrega de terceros, proveedor que cierra el negocio, problemas contractuales

Pruebas Basadas en Riesgos —💡 Comprender (K2)

- **Qué probar:** las áreas de mayor riesgo reciben más atención
- **Cuánto:** las áreas más riesgosas reciben más casos de prueba
- **Cuándo:** los ítems de mayor riesgo se prueban antes
- **Dos etapas:** (1) Identificación de riesgos — tormenta de ideas sobre qué podría salir mal; (2) Evaluación de riesgos — evaluar la probabilidad y el impacto, luego priorizar

5.2.4 Control de Riesgo de Producto —💡 Comprender (K2)

- **Mitigación de riesgos:** probar más exhaustivamente las áreas de alto riesgo; usar técnicas específicas; aplicar múltiples actividades independientes a los ítems de alto riesgo
- **Monitoreo de riesgos:** hacer seguimiento de si la mitigación redujo el riesgo; identificar nuevos riesgos emergentes; actualizar el registro de riesgos

5.9 Monitoreo, Control y Cierre de Pruebas —💡 Comprender (K2)

Monitoreo de Pruebas —💡 Comprender (K2)

Actividad continua de verificar el progreso de las pruebas con respecto al plan.

Métricas de prueba comúnmente recopiladas:

Métrica	Qué indica
% de casos de prueba ejecutados	Cuánto del plan está completado
% aprobados / fallidos / bloqueados	Señal de calidad
Tasa de detección de defectos	Cuántos defectos se están encontrando con el tiempo
Densidad de defectos por módulo	Dónde se concentran los defectos
Cobertura de requisitos / código	Qué tan exhaustivamente se ejercita el sistema
Consumo de recursos / presupuesto	¿Está el proyecto en camino?

Control de Pruebas —💡 Comprender (K2)

Tomar acciones correctivas cuando el monitoreo revela desviaciones con respecto al plan.

- Repriorizar pruebas cuando se encuentra un defecto crítico
- Ajustar el cronograma o el alcance si los riesgos se materializan
- Cambiar el enfoque de prueba o agregar recursos

El monitoreo sin control es solo observación. El control es lo que hace útil al monitoreo.

Informes de Progreso de Pruebas — 💡 Comprender (K2)

- Período cubierto; progreso con respecto al plan; impedimentos/bloqueos; próximas actividades planificadas

Informes de Cierre de Pruebas — 💡 Comprender (K2)

- Qué se probó, cómo y con qué resultados; desviaciones del plan; estado de defectos; testware a archivar; lecciones aprendidas

5.3.3 Comunicación del Estado de las Pruebas — 💡 Comprender (K2)

Opción	Cuándo usarla
Comunicación verbal	Actualizaciones informales e inmediatas; útil para la colaboración diaria
Dashboards	Resúmenes visuales en tiempo real del progreso de las pruebas, la cobertura y el estado de los defectos; adecuados para equipos distribuidos y ejecutivos
Comunicación electrónica	Correos electrónicos o mensajería automatizada para eventos específicos (p. ej., ejecución de prueba completada, defecto crítico encontrado)
Informes de progreso de pruebas	Informes periódicos formales durante la ejecución de pruebas; hace seguimiento del progreso con respecto al plan
Informes de cierre de pruebas	Producidos en los hitos principales (fin de un nivel de prueba, fin de iteración, release); incluye resumen de lo que se probó, resultados, riesgos residuales

Consejo para el examen (K2): Una pregunta puede describir la necesidad de información de una parte interesada y preguntar qué canal de comunicación es más apropiado. Relaciona la formalidad y la frecuencia del canal con las necesidades de la parte interesada.

5.4 Gestión de la Configuración — 💡 Comprender (K2)

La gestión de la configuración identifica, controla y hace seguimiento de los productos de trabajo y las versiones a lo largo del ciclo de vida del proyecto.

- **Ítem de configuración:** cualquier producto de trabajo bajo gestión de la configuración (casos de prueba, scripts, datos, especificaciones del entorno, planes)
- **Línea base:** versión aprobada formalmente de un ítem — un estado conocido y estable; los cambios requieren control de cambios formal
- **Gestión de la configuración y pruebas:** garantiza que los testers usen las versiones correctas; permite la reproducibilidad; apoya el análisis de impacto

Un buen informe de defectos incluye — Aplicar (K3)

Campo	Propósito
ID único	Seguimiento
Resumen	Descripción breve del fallo
Fecha y autor	Quién lo encontró y cuándo
Entorno	Sistema operativo, navegador, versión, configuración
Pasos para reproducir	Secuencia exacta para reproducir el fallo
Resultado esperado	Lo que debería haber ocurrido
Resultado real	Lo que ocurrió realmente
Severidad	Impacto en el sistema (Crítico, Mayor, Menor, Trivial)
Prioridad	Con qué urgencia debe corregirse
Estado	Estado actual en el ciclo de vida
Adjuntos	Logs, capturas de pantalla, vídeos
Contexto del defecto	Fase de prueba, entorno, técnica de prueba utilizada cuando se encontró el defecto
Referencias	Vínculos al caso de prueba que encontró el defecto, requisitos relacionados o defectos relacionados

Severidad vs. Prioridad: Un error tipográfico en la etiqueta de un botón de inicio de sesión es de baja severidad pero posiblemente de alta prioridad (es lo primero que ve cada usuario). Un fallo en una función de uso infrecuente puede ser de alta severidad pero de baja prioridad.

Término	Definición
Plan de prueba	Documento que describe el alcance, el enfoque, el cronograma y los recursos para las pruebas
Criterios de entrada	Condiciones que deben cumplirse antes de que comience una actividad de prueba
Criterios de salida	Condiciones que determinan cuándo una actividad de prueba está completa
Riesgo	Un evento potencial, peligro, amenaza o situación cuya ocurrencia causa un efecto adverso
Riesgo de producto	Riesgo de que un componente del sistema no cumpla con el comportamiento esperado
Riesgo de proyecto	Riesgo para el cronograma, el presupuesto o los recursos del proyecto
Pruebas basadas en riesgos	Usar el riesgo como guía para priorizar qué, cuánto y cuándo probar
Pirámide de pruebas	Modelo que recomienda más pruebas unitarias que pruebas de integración o de UI; el número y los nombres de las capas pueden variar
Cuadrantes de prueba	Marco que categoriza las pruebas por audiencia (negocio vs. tecnología) y propósito (guiar vs. criticar)
Monitoreo de pruebas	Verificar el progreso de las pruebas con respecto al plan
Control de pruebas	Tomar acciones correctivas cuando se detectan desviaciones
Informe de defectos	Documento que captura un fallo y toda la información necesaria para reproducirlo, hacerle seguimiento y resolverlo
Severidad	El impacto de un defecto en el sistema
Prioridad	La urgencia con la que debe corregirse un defecto
Gestión de la configuración	Seguimiento de versiones del testware para garantizar consistencia y reproducibilidad
Wideband Delphi	Técnica de estimación basada en expertos que usa rondas anónimas de estimación y discusión hasta alcanzar el consenso
Estimación de tres puntos	Estimación que usa valores optimista, más probable y pesimista: $(a + 4m + b) / 6$
Mitigación de riesgos	Acciones tomadas para reducir la probabilidad o el impacto de los riesgos identificados
Monitoreo de riesgos	Hacer seguimiento de los riesgos a lo largo del proyecto y actualizar el registro de riesgos en consecuencia

Preguntas de Práctica

- ¿Cuál es el propósito de un plan de prueba y qué incluye normalmente?
- ¿Cuál es la diferencia entre criterios de entrada y criterios de salida? Da un ejemplo de cada uno.

- ¿Cuáles son las cuatro técnicas de estimación nombradas en CTFL v4.0.1? ¿Cómo funciona la estimación de tres puntos?
- ¿Cuál es la relación entre Planning Poker y Wideband Delphi?
- ¿Qué es la priorización de pruebas basada en riesgos y por qué es la estrategia más común?
- ¿Qué es la pirámide de pruebas y qué recomienda sobre la automatización? ¿Por qué puede variar el número de capas?
- ¿Cuáles son los cuatro cuadrantes de prueba y qué ejes los definen?
- ¿A qué cuadrante pertenece la prueba de usabilidad y por qué es importante?
- ¿Cuál es la diferencia entre riesgo de producto y riesgo de proyecto?
- ¿Cuáles son las cuatro categorías de riesgo de proyecto nombradas en el programa del curso?
- ¿Cuáles son las dos etapas de las pruebas basadas en riesgos (identificación y evaluación)?
- ¿Cuáles son las dos actividades del control de riesgo de producto?
- ¿Cuál es la diferencia entre monitoreo de pruebas y control de pruebas?
- ¿Qué debe incluir un informe de cierre de pruebas?
- ¿Cuáles son las cinco opciones de comunicación para informar el estado de las pruebas? ¿Cuándo usarías un dashboard en lugar de una actualización verbal?
- ¿Cuál es la diferencia entre severidad y prioridad en la gestión de defectos?
- ¿Por qué es importante la gestión de la configuración para las pruebas? ¿Qué es una línea base?
- ¿Cuáles son los dos campos añadidos a los informes de defectos más allá de la lista básica (contexto y referencias)?
- ¿Cómo se relacionan los criterios de entrada/salida con la Definition of Ready y la Definition of Done en Agile?
- ¿Qué métricas recopilarías para monitorear el progreso de las pruebas?

Capítulo 6 - Herramientas de Prueba

La idea central: La herramienta adecuada hace que las pruebas sean más rápidas, más consistentes y más exhaustivas — pero las herramientas no son una solución mágica. Conocer qué categorías de herramientas existen, qué hacen realmente y cómo elegir las con criterio es de lo que trata este capítulo.

Niveles Cognitivos en Este Capítulo

K1 = Recordar · K2 = Comprender · K3 = Aplicar — ver Capítulo 1 para la leyenda completa.

Objetivos de Aprendizaje del Capítulo 6

LO	K	Tema
FL-6.1.1	K2	Explicar cómo los diferentes tipos de herramientas de prueba apoyan las pruebas
FL-6.2.1	K1	Recordar los beneficios y riesgos de la automatización de pruebas

Consejo para el examen: El Capítulo 6 es el capítulo más corto y **no tiene K3**. El nivel más alto es K2 (FL-6.1.1). FL-6.2.1 es K1 — recuerdo, no explicación. No inviertas demasiado tiempo aquí.

6.1 Soporte de Herramientas para las Pruebas — Comprender (K2)

Las herramientas se clasifican por la **actividad que apoyan** — muchas herramientas modernas cubren múltiples categorías.

Punto clave para el examen (FL-6.1.1): Debes poder relacionar una categoría de herramienta con la actividad de prueba que apoya. El formato de pregunta es típicamente: "¿qué tipo de herramienta apoyaría la actividad X?"

Herramientas de Gestión e Informes — 💡 Comprender (K2)

Tipo de herramienta	Qué apoya
Herramientas de gestión de pruebas	Planificación, monitoreo y control de actividades de prueba; gestión de casos de prueba, suites de prueba y cronogramas de ejecución
Herramientas de seguimiento de defectos	Registro, seguimiento y gestión de defectos a lo largo de su ciclo de vida
Herramientas de gestión de la configuración	Seguimiento de versiones del software bajo prueba y del testware; garantiza la reproducibilidad
Herramientas de gestión de requisitos	Trazabilidad entre requisitos y casos de prueba

Herramientas de Pruebas Estáticas — 💡 Comprender (K2)

Tipo de herramienta	Qué apoya
Herramientas de análisis estático	Analizar el código fuente sin ejecutarlo — detectar violaciones de estándares de codificación, vulnerabilidades de seguridad, código inalcanzable, variables no definidas
Correctores ortográficos y gramaticales	Revisar los productos de trabajo escritos (requisitos, casos de prueba) en cuanto a calidad del lenguaje

Herramientas de Diseño e Implementación de Pruebas — 💡 Comprender (K2)

Tipo de herramienta	Qué apoya
Herramientas de generación de datos de prueba	Crear datos de entrada para las pruebas automáticamente — útil para valores límite, grandes conjuntos de datos y datos de producción enmascarados
Herramientas de diseño de casos de prueba	Generar casos de prueba a partir de modelos o especificaciones (usadas en pruebas basadas en modelos)
Herramientas de pruebas basadas en modelos	Derivar automáticamente casos de prueba a partir de un modelo de comportamiento del sistema

Herramientas de Ejecución y Cobertura de Pruebas — 💡 Comprender (K2)

Tipo de herramienta	Qué apoya
Herramientas de ejecución de pruebas (frameworks de automatización de pruebas)	Ejecutar scripts de prueba automáticamente; comparar resultados reales con resultados esperados; registrar resultados
Herramientas de medición de cobertura	Medir cuánto del código o de los requisitos ha sido ejercitado por las pruebas (p. ej., cobertura de sentencias, cobertura de ramas)

- **Pruebas dirigidas por datos:** el script de prueba está separado de los datos de prueba (hoja de cálculo, base de datos) — el mismo script, muchos conjuntos de datos
- **Pruebas dirigidas por palabras clave:** la lógica de prueba se expresa como palabras clave legibles (p. ej., `Login` , `ClickButton`) — menos técnica para mantener

Herramientas de Pruebas No Funcionales — 💡 Comprender (K2)

Tipo de herramienta	Qué apoya
Herramientas de pruebas de rendimiento	Simular carga, medir tiempos de respuesta, identificar cuellos de botella (pruebas de carga, pruebas de estrés, pruebas de resistencia)
Herramientas de pruebas de seguridad	Escanear vulnerabilidades, probar autenticación y autorización, soporte para pruebas de penetración
Herramientas de pruebas de accesibilidad	Verificar que el sistema pueda ser utilizado por personas con discapacidades

Herramientas de DevOps y Colaboración — 💡 Comprender (K2)

Tipo de herramienta	Qué apoya
Herramientas de pipeline CI/CD	Activar automáticamente la ejecución de pruebas en cada commit de código; integrar las pruebas en el proceso de construcción
Herramientas de control de versiones / SCM	Gestionar versiones del código fuente y del testware; habilitar el desarrollo paralelo y por ramas
Herramientas de colaboración	Comunicación, documentación de pruebas compartida, coordinación de equipos remotos y toma de decisiones compartida

Herramientas de Escalabilidad y Estandarización del Despliegue — 💡 Comprender (K2)

Tipo de herramienta	Qué apoya
Máquinas virtuales y herramientas de contenedorización (p. ej., Docker, Kubernetes)	Apoyar la escalabilidad y el aprovisionamiento consistente de entornos de prueba entre equipos y pipelines

Herramientas de Propósito General — 💡 Comprender (K2)

Tipo de herramienta	Qué apoya
Herramientas de propósito general (p. ej., hojas de cálculo)	Cualquier otra herramienta que ayude en las pruebas — las hojas de cálculo son una herramienta de gestión de pruebas válida, aunque básica, para proyectos pequeños

Recordar (K1)

La automatización es más valiosa para las **pruebas repetitivas, estables y de alto volumen** — no para todo.

Principio clave: Adquirir una herramienta simplemente no garantiza el éxito. Cada nueva herramienta requiere esfuerzo para lograr beneficios reales y duraderos — incluyendo la introducción de la herramienta, el mantenimiento continuo y la formación del equipo.

Beneficios de la Automatización de Pruebas — 🧠 Recordar (K1)

Beneficio	Por qué es importante
Reduce el trabajo manual repetitivo	Libera a los testers para enfocarse en pruebas exploratorias y de mayor valor
Previene errores humanos simples	Un script se ejecuta exactamente de la misma manera cada vez
Evaluación objetiva	Las métricas de cobertura y los resultados se miden de forma consistente
Acceso fácil a información del estado de las pruebas	Los dashboards de resultados y los logs se generan automáticamente
Reducción del tiempo de ejecución de pruebas	Las pruebas pueden ejecutarse en paralelo, durante la noche o en cada commit
Más tiempo para que los testers diseñen pruebas nuevas, más profundas y más efectivas	La automatización se encarga del trabajo de regresión repetitivo, liberando a los testers para invertir en un mejor diseño de pruebas

Riesgo	Qué puede salir mal
Expectativas poco realistas	Las herramientas no encuentran bugs automáticamente — solo comprueban lo que se les indica que comprueben
Esfuerzo subestimado	Automatizar pruebas lleva un tiempo significativo para configurar, escribir scripts y mantener
Usar la herramienta equivocada	No toda herramienta se adapta a cada stack tecnológico o tipo de prueba
Exceso de dependencia de la automatización	Las pruebas automatizadas no pueden reemplazar el juicio humano, especialmente para UX y pruebas exploratorias
Carga de mantenimiento	Cuando el sistema cambia, los scripts de prueba se rompen y deben actualizarse
Dependencia del proveedor	El proveedor de la herramienta puede cerrar el negocio, retirar la herramienta, venderla a un proveedor diferente o brindar soporte deficiente
Abandono del código abierto	Una herramienta de código abierto puede ser abandonada, sin más actualizaciones ni parches de seguridad
Incompatibilidad de plataforma	La herramienta de automatización puede no ser compatible con la plataforma de desarrollo, el framework o la tecnología bajo prueba
Incumplimiento normativo	La herramienta elegida puede no cumplir con los requisitos regulatorios o los estándares de seguridad aplicables al proyecto

La clave: La automatización es un multiplicador — multiplica la eficiencia de las buenas pruebas, pero también multiplica los problemas de las malas pruebas. Las malas pruebas automatizadas a escala son peores que no tener automatización.

Término	Definición
Automatización de pruebas	Usar herramientas para ejecutar pruebas y comparar los resultados reales con los esperados automáticamente
Framework de automatización de pruebas	Un conjunto de reglas, bibliotecas y herramientas que proporcionan la estructura para los scripts de prueba automatizados
Pruebas dirigidas por datos	Un enfoque de automatización de pruebas donde los datos de prueba se almacenan externamente y se alimentan a los scripts; el mismo script se ejecuta con diferentes conjuntos de datos
Pruebas dirigidas por palabras clave	Un enfoque de automatización de pruebas donde las acciones de prueba se expresan como palabras clave legibles, separando la lógica de prueba de la implementación
Herramienta de análisis estático	Una herramienta que analiza el código fuente sin ejecutarlo, detectando defectos y violaciones de estándares
Herramienta de gestión de pruebas	Una herramienta que apoya la planificación, el monitoreo, el control y la elaboración de informes sobre las actividades de prueba
Herramienta de pruebas de rendimiento	Una herramienta que simula carga y mide los tiempos de respuesta del sistema y su comportamiento bajo estrés
Pruebas basadas en modelos	Un enfoque donde los casos de prueba se generan automáticamente a partir de un modelo de comportamiento del sistema
CI/CD	Continuous Integration / Continuous Delivery — una práctica DevOps donde los cambios de código activan construcciones y pruebas automatizadas
Herramienta de medición de cobertura	Una herramienta que mide cuánto del código o de los requisitos es ejercitado por una suite de pruebas

Preguntas de Práctica

- ¿Cuáles son las principales categorías de herramientas de prueba? Da un ejemplo de lo que apoya cada una.
- ¿Cuál es la diferencia entre las pruebas dirigidas por datos y las pruebas dirigidas por palabras clave?
- Nombra tres beneficios y tres riesgos de la automatización de pruebas.
- ¿Por qué la automatización de pruebas requiere un esfuerzo de mantenimiento continuo?
- ¿Qué factores deben considerarse al seleccionar una herramienta de prueba?
- ¿Qué es una prueba de concepto en el contexto de la selección de herramientas y por qué se recomienda?
- ¿Por qué una suite de pruebas automatizadas que pasa no garantiza la calidad del software?
- ¿Cuál es la diferencia entre una herramienta de análisis estático y una herramienta de ejecución de pruebas?
- ¿Cómo se conectan las herramientas CI/CD con el concepto de desplazamiento hacia la izquierda del Capítulo 2?

- ¿Qué tipo de herramienta usarías para medir la cobertura de ramas después de ejecutar tu suite de pruebas?

ISTQB CTFL — Glosario

Todos los términos clave de los Capítulos 1–6, ordenados alfabéticamente. Cada entrada muestra el/los capítulo(s) donde se evalúa y el nivel cognitivo.

Cómo usarlo: Lee el término, intenta recordar la definición y luego verifícala. Dedica tiempo extra a los términos K2 (necesitas *explicarlos*, no solo recordarlos) y K3 (necesitas *aplicarlos*).

A

Acceptance criteria Ch4  K2 Las condiciones que una historia de usuario debe cumplir para ser aceptada por los interesados. Definen el alcance de la historia, guían las pruebas de aceptación y, generalmente, se redactan en formato orientado a escenarios (Given/When/Then) o en formato orientado a reglas.

Acceptance testing Ch2  K2 El nivel de prueba enfocado en la validación — confirmar que el sistema está listo para su despliegue. Lo realizan representantes de negocio o usuarios finales. Incluye UAT, OAT, pruebas contractuales/regulatorias, pruebas alfa y pruebas beta.

Alpha testing Ch2  K2 Un tipo de prueba de aceptación realizada por usuarios internos en las instalaciones del desarrollador, antes del lanzamiento.

Anomaly Ch3  K1 Cualquier resultado o condición que se desvía de las expectativas, descubierto durante una revisión. Puede ser o no un defecto — debe investigarse.

ATDD — Acceptance Test-Driven Development Ch2 Ch4  K3 Un enfoque en el que los casos de prueba se derivan de los criterios de aceptación como parte del proceso de diseño del sistema, antes de que la historia de usuario sea implementada. Pasos: taller de especificación → creación de casos de prueba → implementación. Los casos de prueba deben cubrir todas las características de la historia de usuario y no deben ir más allá de ella. Está estrechamente relacionado con BDD.

B

BDD — Behavior-Driven Development Ch2  K1 Un enfoque de prueba primero (test-first) donde las pruebas se escriben en lenguaje natural usando el formato Given/When/Then, haciéndolas comprensibles para todo el equipo, incluidas las partes interesadas no técnicas.

Beta testing Ch2  K2 Un tipo de prueba de aceptación realizada por usuarios externos en su propio entorno, después del lanzamiento.

Black-box technique Ch4  K2 Una técnica de prueba que deriva los casos de prueba del comportamiento especificado del software — sin conocimiento de su estructura interna. Las pruebas sobreviven a los cambios de código porque están basadas en especificaciones. Véase también: *Equivalence Partitioning*, *Boundary Value Analysis*, *Decision Table Testing*, *State Transition Testing*.

Black-box testing Ch2  K2 Un tipo de prueba en el que las pruebas se basan en la especificación o el comportamiento especificado del sistema, sin conocimiento de la estructura interna. Puede aplicarse en

cualquier nivel de prueba.

Boundary Value Analysis (BVA)   Una técnica de caja negra que extiende la partición de equivalencia al apuntar específicamente a los límites de las particiones ordenadas. Dos versiones: BVA de 2 valores (límite + vecino más cercano) y BVA de 3 valores (límite + valor inmediatamente inferior + valor inmediatamente superior). Aprovecha el hecho de que los desarrolladores cometen más errores en los extremos de los rangos.

Branch   Una transferencia de control entre nodos en un grafo de flujo de control. Las ramas pueden ser incondicionales o condicionales (por ejemplo, de una sentencia if/else o switch).

Branch coverage   El porcentaje de ramas ejercidas por un conjunto de pruebas. La cobertura del 100% de ramas garantiza la cobertura del 100% de sentencias (subsunción), pero no a la inversa. Es más robusta que la cobertura de sentencias.

Branch testing   Una técnica de caja blanca que diseña casos de prueba para ejercer todas las ramas del código. Corresponde a la cobertura de ramas.

C

Checklist-based testing   Una técnica basada en la experiencia en la que las pruebas son guiadas por una lista preparada de condiciones, frecuentemente formuladas como preguntas. Las listas de verificación deben actualizarse regularmente en base al análisis de defectos para evitar que pierdan efectividad.

Component integration testing   Un nivel de prueba enfocado en probar las interfaces e interacciones entre componentes integrados. Generalmente lo realizan desarrolladores o testers después de las pruebas de componentes.

Component testing   El nivel de prueba enfocado en componentes individuales probados de forma aislada. También llamado prueba unitaria. Típicamente lo realizan los desarrolladores.

Confirmation testing   Prueba realizada después de que un defecto ha sido corregido, para verificar que ese defecto específico ya no causa una falla. También llamada re-testing.

Configuration baseline   Una versión formalmente aprobada de un elemento de configuración, a partir de la cual se controlan los cambios. Representa un estado conocido y estable de un producto de trabajo. Cualquier cambio en una línea base debe pasar por un proceso formal de control de cambios.

Configuration item   Cualquier producto de trabajo sometido a gestión de configuración — por ejemplo, planes de prueba, casos de prueba, scripts de prueba, datos de prueba o especificaciones del entorno de prueba.

Configuration management (CM)   La disciplina de identificar, controlar y rastrear los productos de trabajo y sus versiones a lo largo del ciclo de vida del proyecto. Garantiza que los testers siempre trabajen con la versión correcta de la base de prueba y el testware. Permite la reproducibilidad de los resultados de prueba y apoya el análisis de impacto. Conceptos clave: elemento de configuración, línea base, control de cambios.

CI/CD — Continuous Integration / Continuous Delivery   Una práctica DevOps en la que cada cambio de código desencadena automáticamente una compilación y una ejecución de pruebas. Integra las pruebas directamente en el pipeline de desarrollo, lo que permite una retroalimentación rápida y apoya el desplazamiento a la izquierda (shift left). Las herramientas CI/CD conectan los commits de los desarrolladores con los resultados de las pruebas sin intervención manual.

Coverage measurement tool   Una herramienta que mide cuánto del código o de los requisitos ha sido ejercido por un conjunto de pruebas. Produce métricas objetivas como los porcentajes de cobertura de sentencias o de ramas.

Coverage criterion   Una regla o meta utilizada para determinar si un conjunto de pruebas es suficiente. Ejemplos: todas las particiones ejercidas (EP), todas las ramas ejercidas (cobertura de ramas), todas las transiciones válidas ejercidas (transición de estados).

D

Data-driven testing   Un enfoque de automatización de pruebas en el que el script de prueba se separa de los datos de prueba. Los datos se almacenan externamente (por ejemplo, en una hoja de cálculo o base de datos) y se introducen en el script en tiempo de ejecución, lo que permite que el mismo script se ejecute con muchos conjuntos de datos diferentes.

Decision table testing   Una técnica de caja negra utilizada para probar combinaciones de condiciones y sus acciones resultantes. Cada columna de la tabla representa una regla (combinación única de condiciones). Es más adecuada para probar lógica de negocio con comportamiento condicional complejo.

Defect   Un fallo o imperfección en un producto de trabajo (código, documento, diseño) introducido por un error humano. Cuando se ejecuta, un defecto puede causar una falla — o no, dependiendo de las condiciones.

Defect management   El proceso de rastrear los defectos desde su descubrimiento hasta su cierre. Implica registrar, clasificar, priorizar, corregir, volver a probar y cerrar los defectos. El examen evalúa el contenido de un informe de defecto (FL-5.4.1/5.4.2).

Defect report   Un documento que registra un defecto, que contiene toda la información necesaria para reproducirlo, rastrearlo y resolverlo. Campos clave: ID único, resumen, fecha/autor, entorno, pasos para reproducir, resultado esperado, resultado real, severidad, prioridad, estado y adjuntos.

Definition of Done (DoD)   Equivalente ágil de los criterios de salida — las condiciones que deben cumplirse para que una historia de usuario o iteración se considere completa.

Definition of Ready (DoR)   Equivalente ágil de los criterios de entrada — las condiciones que deben cumplirse antes de que pueda comenzar el trabajo en una historia de usuario o iteración.

Dynamic testing    Prueba realizada ejecutando el software. Los defectos se revelan a través de las fallas — el sistema debe ejecutarse. Contrasta con las pruebas estáticas.

E

Entry criteria   Condiciones que deben satisfacerse antes de que comience una actividad de prueba. Evita el desperdicio de esfuerzo en pruebas cuando el sistema no está listo. En Agile: Definition of Ready (DoR).

Equivalence partitioning (EP)   Una técnica de caja negra que divide los datos de entrada (o salida) en particiones donde se espera que todos los valores sean tratados de forma idéntica por el sistema. Un caso de prueba por partición es suficiente. Cobertura = particiones ejercidas ÷ total de particiones. Las particiones pueden ser continuas o discretas, ordenadas o no ordenadas, finitas o infinitas. Las fuentes incluyen no solo parámetros de entrada, sino también elementos de configuración, valores internos, valores relacionados con el tiempo y parámetros de interfaz. Las particiones deben ser no vacías y no superpuestas.

Estimation based on ratios    Una técnica de estimación del esfuerzo de prueba que utiliza métricas de proyectos similares anteriores — por ejemplo, la proporción del esfuerzo de prueba respecto al esfuerzo de desarrollo — y aplica esas proporciones a los datos del proyecto actual.

Extended-entry decision table    Una variante de la tabla de decisión en la que las condiciones pueden tomar múltiples valores (rangos, particiones o valores discretos) en lugar de solo verdadero/falso. Las acciones también pueden especificar valores de salida exactos. Más compacta que las tablas de entrada limitada, pero más difícil de leer. Contrasta con: *tabla de decisión de entrada limitada*.

Extrapolation    Una técnica de estimación del esfuerzo de prueba que utiliza mediciones recopiladas al principio del proyecto actual para estimar el esfuerzo restante. Cuantos más puntos de datos se recopilen, más precisa será la estimación.

Error    Un error humano — por ejemplo, un desarrollador que malinterpreta un requisito. Un error introduce un defecto en un producto de trabajo.

Error guessing    Una técnica basada en la experiencia que utiliza el conocimiento del tester sobre cómo el software falla habitualmente para anticipar y apuntar a los defectos probables. Puede estructurarse como un *fault attack* usando una lista de errores probables.

Exit criteria    Condiciones que determinan cuándo una actividad de prueba está suficientemente completa. Pueden incluir metas de cobertura, umbrales de densidad de defectos o agotamiento del tiempo/presupuesto. En Agile: Definition of Done (DoD).

Experience-based technique    Una categoría de técnicas de prueba basadas en el conocimiento, la intuición y la experiencia del tester. Complementa las técnicas de caja negra y caja blanca. Incluye error guessing, pruebas exploratorias y pruebas basadas en listas de verificación.

Exploratory testing    Una técnica basada en la experiencia en la que el tester diseña, ejecuta y evalúa las pruebas simultáneamente mientras aprende sobre el sistema. Utiliza un *test charter* para guiar cada sesión con límite de tiempo. Es más eficaz cuando las especificaciones son incompletas o el tiempo es limitado.

F

Failure    El comportamiento incorrecto visible y observable de un sistema causado por la ejecución de un defecto. No todos los defectos causan fallas — algunos requieren condiciones específicas para activarse.

Fault attack   

Una forma estructurada de error guessing: el tester crea una lista de errores y defectos probables y luego diseña pruebas específicamente para exponerlos.

Functional testing    Un tipo de prueba que evalúa *qué* hace el sistema — su completitud funcional, corrección y adecuación. Puede aplicarse en cualquier nivel de prueba.

I

Impact analysis   

El proceso de evaluar qué partes de un sistema se ven afectadas por un cambio propuesto. Se utiliza antes de las pruebas de mantenimiento para definir el alcance de las pruebas de regresión. Depende de una buena

trazabilidad.

Informal review   El tipo de revisión menos formal. Sin proceso definido, sin resultado requerido. El objetivo es simplemente encontrar anomalías a través de una inspección informal.

Inspection   El tipo de revisión más formal. Sigue un proceso definido completo basado en ISO/IEC 20246. La detección máxima de anomalías es el objetivo principal; se recopilan métricas. El autor **no puede** actuar como líder de revisión ni como escribano — exclusivo de este tipo de revisión.

INVEST   Un acrónimo que define las características de una buena historia de usuario: Independiente, **N**egociable, **V**aliosa, **E**stimable, **P**equaña (Small), **V**erificable (Testable).

K

Keyword-driven testing   Un enfoque de automatización de pruebas en el que las acciones de prueba se expresan como palabras clave legibles (por ejemplo, `Login`, `ClickButton`, `VerifyText`). Las palabras clave se implementan por separado, lo que permite a personas no programadoras escribir y mantener pruebas sin conocimiento de scripting.

M

Maintenance testing   Prueba realizada en un sistema ya en producción cuando necesita cambios. Se activa por modificaciones (mejoras, correcciones de errores, hot fixes), actualizaciones/migraciones (nueva plataforma, SO, entorno) o retirada (archivado de datos al fin de vida útil).

Model-based testing   Un enfoque de prueba en el que los casos de prueba se derivan automáticamente de un modelo de comportamiento del sistema. El modelo (por ejemplo, una máquina de estados o una tabla de decisión) sirve como base de prueba; una herramienta genera casos de prueba a partir de él. Reduce el esfuerzo manual de diseño de pruebas para sistemas complejos.

Moderator   El rol responsable de conducir una reunión de revisión de manera efectiva. Maneja la facilitación, mediación y gestión del tiempo, y garantiza un entorno seguro para la retroalimentación. Obligatorio en revisiones técnicas e inspecciones; opcional en walkthroughs.

N

Non-functional testing   Un tipo de prueba que evalúa *qué tan bien* funciona el sistema — cubriendo rendimiento, seguridad, usabilidad, fiabilidad, mantenibilidad, portabilidad y seguridad (safety). Puede aplicarse en cualquier nivel de prueba.

O

OAT — Operational Acceptance Testing   Un tipo de prueba de aceptación enfocada en la preparación operacional: respaldo y recuperación, seguridad, rendimiento bajo carga y otras características operacionales.

P

Planning Poker   Una técnica de estimación ágil en la que los miembros del equipo seleccionan de forma independiente una carta (que representa su estimación) y revelan todas las cartas simultáneamente. Las diferencias se discuten hasta alcanzar el consenso. Planning Poker es una **variante de Wideband Delphi** adaptada para equipos ágiles — no es una técnica de estimación independiente.

Priority   Con qué urgencia debe corregirse un defecto, desde la perspectiva del negocio. Alta prioridad significa que debe corregirse pronto — independientemente de la severidad. Un error cosmético en la pantalla de inicio de sesión puede ser de baja severidad pero alta prioridad.

Product risk   El riesgo de que un componente o funcionalidad del sistema no cumpla con el comportamiento o la calidad esperados. Los riesgos de producto determinan el alcance y la profundidad de las pruebas.

Performance testing tool   Una herramienta que simula carga de usuarios concurrentes en un sistema y mide los tiempos de respuesta, el rendimiento (throughput) y el consumo de recursos bajo estrés. Se utiliza para pruebas de carga, pruebas de estrés y pruebas de resistencia.

Project risk   El riesgo para el propio proyecto — cronograma, presupuesto, recursos o equipo. Se gestiona a través de la gestión de proyectos, no mediante pruebas.

Q

Quality assurance (QA)   Una actividad preventiva orientada al proceso que mejora el proceso de desarrollo para evitar que ocurran defectos. Diferente de las pruebas, que están orientadas al producto y son correctivas. QA establece el buen proceso; las pruebas verifican el resultado.

R

Regression testing   Volver a ejecutar las pruebas después de un cambio para detectar efectos secundarios no deseados — garantizando que la funcionalidad existente no se haya roto. Crece con cada versión y es una candidata sólida para la automatización.

Review   Una forma de prueba estática en la que una o más personas evalúan manualmente un producto de trabajo. Los tipos van desde informal hasta formal (inspección). Objetivo: encontrar anomalías, mejorar la calidad, construir una comprensión compartida.

Review leader   El rol responsable de organizar la revisión en su conjunto: decidir quién participa, cuándo y dónde. No puede ser la misma persona que el escribano o el autor en una inspección.

Risk   Un evento potencial, peligro, amenaza o situación cuya ocurrencia causa un efecto adverso. Se caracteriza por su probabilidad e impacto. Nivel de riesgo = probabilidad × impacto.

Risk assessment   La segunda etapa de las pruebas basadas en riesgo: evaluar los riesgos identificados según su probabilidad e impacto para determinar qué áreas necesitan mayor atención en las pruebas.

Risk identification   La primera etapa de las pruebas basadas en riesgo: generar ideas con los interesados para identificar qué podría salir mal en el sistema.

Risk-based testing   **K2** Un enfoque de prueba que utiliza el riesgo como guía principal para decidir qué probar, cuánto probar y cuándo probar. Garantiza que las áreas de mayor riesgo reciban la mayor atención en las pruebas — especialmente importante cuando el tiempo es limitado.

Root cause   **K1** La razón subyacente fundamental por la que se cometió un error — por ejemplo, un requisito malentendido, falta de capacitación o documentación poco clara. Corregir las causas raíz evita defectos futuros.

S

Scribe   **K1** El rol responsable de registrar anomalías, decisiones y nuevos hallazgos durante una reunión de revisión. Obligatorio en las inspecciones. Opcional en los walkthroughs — el autor puede desempeñar este rol.

Severity  **K2** El impacto de un defecto en el sistema — en qué medida afecta la funcionalidad, los datos o las operaciones. Se clasifica como Crítico, Mayor, Menor o Trivial. No es lo mismo que la prioridad.

Shift left  **K2** La práctica de trasladar las actividades de prueba más temprano en el SDLC — realizando el análisis, diseño e incluso la ejecución de pruebas lo antes posible para encontrar defectos cuando son más baratos de corregir.

SDLC — Software Development Lifecycle   **K1** El proceso utilizado para planificar, crear, probar y entregar software. La elección del modelo SDLC (secuencial, iterativo, incremental) impacta directamente en cómo se estructura la prueba y cuándo ocurre.

State transition testing   **K3** Una técnica de caja negra que prueba el comportamiento de un sistema basado en sus estados y las transiciones entre ellos. Criterios de cobertura: todos los estados (el más débil), todas las transiciones válidas / 0-switch (el más común), todas las transiciones incluyendo las inválidas (el más fuerte).

Statement coverage  **K2** El porcentaje de sentencias ejecutables ejercidas por un conjunto de pruebas. La cobertura del 100% de sentencias no garantiza que todas las ramas sean probadas. Más débil que la cobertura de ramas.

Statement testing  **K2** Una técnica de caja blanca que diseña casos de prueba para ejercer sentencias ejecutables. Corresponde a la cobertura de sentencias.

Static analysis  **K2** Evaluación asistida por herramienta de un producto de trabajo frente a una sintaxis formal — sin ejecutarlo. Ejemplos: linters, analizadores de código. Encuentra problemas como variables no definidas, código inalcanzable e incumplimientos de estándares de codificación. En el Ch6, las herramientas de análisis estático son una categoría de herramientas nombrada que apoya las actividades de prueba estática.

Static testing  **K2** Prueba realizada sin ejecutar el software. Incluye tanto revisiones manuales como análisis estático automatizado. Puede encontrar defectos en productos de trabajo no ejecutables (requisitos, diseños, planes de prueba) a los que las pruebas dinámicas no pueden llegar.

System integration testing  **K2** Un nivel de prueba enfocado en las interfaces entre el sistema bajo prueba y los sistemas o servicios externos. Prueba las interacciones en el límite del sistema.

System testing  **K2** Un nivel de prueba que evalúa el comportamiento de extremo a extremo de todo el sistema frente a sus requisitos especificados. Generalmente lo realizan testers independientes.

Test automation   El uso de herramientas para ejecutar pruebas y comparar resultados reales con los esperados de forma automática, sin intervención manual. Más eficaz para pruebas repetitivas, estables y de alto volumen. No reemplaza el juicio humano ni las pruebas exploratorias.

Test automation framework   Un conjunto de reglas, bibliotecas y herramientas que proporcionan la estructura y las convenciones para escribir, organizar y ejecutar scripts de prueba automatizados. Ejemplos incluyen frameworks basados en datos y basados en palabras clave.

Test management tool   Una herramienta que apoya la planificación, monitoreo, control e informes de las actividades de prueba. Generalmente gestiona casos de prueba, suites de prueba, calendarios de ejecución de pruebas y seguimiento de defectos en un solo lugar.

TDD — Test-Driven Development   Una práctica de desarrollo en la que los desarrolladores primero escriben una prueba unitaria que falla, luego escriben código para que pase y después refactorizan. Implementa el principio de prueba temprana.

Technical review   Un tipo de revisión semiformal dirigido por un moderador capacitado con revisores técnicamente calificados. El enfoque es alcanzar consenso sobre cuestiones técnicas y detectar anomalías. Los objetivos incluyen alcanzar consenso sobre problemas técnicos, generar nuevas ideas, evaluar la calidad y generar confianza en el producto de trabajo.

Test basis   Toda la información utilizada para derivar casos de prueba — requisitos, historias de usuario, diseños, código, evaluaciones de riesgo, etc. Una buena trazabilidad vincula los casos de prueba con la base de prueba.

Test charter   Un documento utilizado en las pruebas exploratorias para definir los objetivos de una sesión de prueba con límite de tiempo. Guía la exploración sin restringirla en exceso.

Test completion report   Un informe producido al final de una fase de prueba o proyecto. Resume qué se probó, los resultados, las desviaciones del plan, el estado de los defectos, el testware a archivar y las lecciones aprendidas.

Test control   La actividad de tomar acciones correctivas cuando el monitoreo de pruebas revela desviaciones del plan de prueba — como repriorizar pruebas, ajustar el alcance o agregar recursos.

Test level   Un grupo de actividades de prueba organizadas y gestionadas conjuntamente, con un enfoque específico y una base de prueba. Los cinco niveles: componente, integración de componentes, sistema, integración de sistemas y prueba de aceptación.

Test monitoring   La actividad continua de verificar el progreso real de las pruebas frente al plan, utilizando métricas. Permite el control de pruebas cuando se encuentran desviaciones.

Test object   El producto de trabajo específico que se está probando — por ejemplo, un componente, un sistema, un documento o una configuración.

Test plan   Un documento que describe los objetivos, el alcance, el enfoque, el cronograma y los recursos para un esfuerzo de prueba. Es un documento vivo que se actualiza a medida que el proyecto evoluciona.

Test progress report   Un informe que comunica el estado actual de las pruebas a los interesados. Cubre el período, el progreso frente al plan, los impedimentos y los próximos pasos planificados.

Test pyramid   Un modelo que recomienda una distribución de pruebas entre niveles: muchas pruebas rápidas y económicas en la base (más cerca del código), menos en los niveles intermedios y muy pocas pruebas lentas/frágiles en la cima (más cerca de la interfaz de usuario). Fomenta la automatización en

el nivel efectivo más bajo. **El número y la denominación de las capas pueden variar** — el principio (más pruebas en los niveles inferiores) importa más que cualquier estructura de capas específica.

Three-point estimation Ch5  K2 Una técnica de estimación del esfuerzo de prueba que produce tres estimaciones: la más optimista (a), la más probable (m), la más pesimista (b). Estimación final = $(a + 4m + b) / 6$. Reduce el sesgo y tiene en cuenta la incertidumbre al ponderar el valor más probable cuatro veces.

Test type Ch2  K2 Una clasificación de las pruebas según qué aspecto del sistema se está probando: funcional, no funcional, caja negra o caja blanca. Los cuatro tipos pueden aplicarse en cualquier nivel de prueba.

Testware Ch1  K2 Todos los artefactos producidos por las actividades de prueba — incluidos planes de prueba, casos de prueba, scripts de prueba, datos de prueba, registros de prueba, informes de defectos e informes de finalización.

Testing quadrants Ch5  K2 Un marco (Brian Marick) para categorizar los tipos de prueba por dos ejes: orientadas a la tecnología vs. orientadas al negocio, y que apoyan al equipo vs. que critican el producto. Q1 (tecnología, soporte): pruebas de componentes/integración de componentes. Q2 (negocio, soporte): pruebas funcionales, pruebas de historia, pruebas de API. Q3 (negocio, crítica): pruebas exploratorias, pruebas de usabilidad, UAT. Q4 (tecnología, crítica): pruebas de rendimiento, carga, estrés, seguridad, smoke — **excepto las pruebas de usabilidad**, que pertenecen a Q3.

U

UAT — User Acceptance Testing Ch2  K2 Un tipo de prueba de aceptación en la que usuarios reales validan que el sistema cumple con sus necesidades y está listo para su uso en producción.

User story Ch4  K2 Una descripción de una funcionalidad que aporta valor a un usuario o comprador. Se escribe como: "*Como [rol], quiero [objetivo], para que [valor de negocio].*" Tiene tres componentes: Tarjeta, Conversación, Confirmación (las 3 C). Debe seguir los criterios INVEST.

V

Validation Ch1  K1 Confirmar que un producto satisface las necesidades del usuario y de otros interesados. La pregunta es: "*¿Estamos construyendo el producto correcto?*" Se enfoca en la aptitud para el propósito.

Verification Ch1  K1 Confirmar que un producto de trabajo cumple con sus requisitos especificados. La pregunta es: "*¿Estamos construyendo el producto correctamente?*" Se enfoca en la conformidad con las especificaciones.

W

Walkthrough Ch3  K1 Un tipo de revisión dirigido por el autor, quien guía a los participantes a través del producto de trabajo. Tiene como objetivo educar, alcanzar consenso y detectar anomalías. La revisión individual previa **no es obligatoria**. Un escribano es **opcional** (el autor puede desempeñar este rol). Los objetivos incluyen: evaluar la calidad, generar confianza, educar a los revisores, alcanzar consenso, generar nuevas ideas, motivar a los autores.

Wideband Delphi Ch5  K2 Una técnica de estimación nombrada en CTFL v4.0.1 basada en el consenso de expertos. Los expertos estiman el esfuerzo de forma independiente, los resultados se comparten de forma anónima, se discuten las diferencias y el proceso se repite hasta alcanzar el consenso. Planning Poker es una variante de Wideband Delphi adaptada para equipos ágiles.

White-box technique Ch4  K2 Una técnica de prueba que deriva los casos de prueba de la estructura interna del software — rutas de código, ramas, flujos de datos. Las pruebas dependen de la implementación. Incluye pruebas de sentencias y pruebas de ramas.

White-box testing Ch2  K2 Un tipo de prueba en el que las pruebas se basan en la estructura interna o la implementación del sistema. Puede aplicarse en cualquier nivel de prueba.

ISTQB CTFL — Errores Comunes

Trampas típicas del examen, conceptos fácilmente confundidos y consejos para evitar caer en ellas.

Conceptos Fácilmente Confundidos

Error vs. Defect vs. Failure vs. Root Cause Ch1

Término	Qué es	Quién lo genera
Error	Un error humano	Desarrollador, analista, tester
Defect	El fallo que el error introdujo en el producto de trabajo	Causado por el error
Failure	El comportamiento incorrecto visible cuando el defecto se ejecuta	Causado por el defecto
Root cause	La razón subyacente por la que se cometió el error	Problema del proceso o del entorno

Trampa: "Un defecto siempre causa una falla." — FALSO. Un defecto puede no ejecutarse nunca, o puede fallar solo bajo condiciones específicas.

Testing vs. Quality Assurance Ch1

	Testing	QA
Enfoque	Producto	Proceso
Objetivo	Encontrar defectos	Prevenir defectos
Orientación	Correctiva	Preventiva

Trampa: "QA y testing son lo mismo." — FALSO. El testing es parte de QA, pero QA es más amplio — abarca todo el proceso de desarrollo.

Verification vs. Validation Ch1

- Verification** → "¿Estamos construyendo el producto **correctamente**?" — conformidad con la especificación
- Validation** → "¿Estamos construyendo el producto **correcto**?" — aptitud para las necesidades del usuario

Consejo para recordarlo: Validation = Valor para el usuario.

Confirmation Testing vs. Regression Testing Ch2

- **Confirmation testing:** vuelve a probar un defecto específico que fue corregido — ¿funcionó *esta* corrección?
- **Regression testing:** vuelve a probar todo el sistema después de cualquier cambio — ¿se rompió algo *más*?

Trampa: "La regression testing solo es necesaria después de correcciones de errores." — FALSO. Cualquier cambio (mejora, refactorización, cambio de configuración) puede introducir regresiones.

Black-box / White-box como Test Types vs. Test Techniques Ch2 Ch4

Los mismos términos aparecen en dos capítulos diferentes con significados relacionados pero distintos:

Contexto	Significado
Ch2 — Test types	<i>En qué</i> se basan las pruebas: especificación (black-box) vs. estructura (white-box)
Ch4 — Test techniques	<i>Cómo</i> se derivan los casos de prueba: desde la especificación (técnicas de caja negra) vs. desde el código (técnicas de caja blanca)

Trampa: Ver "black-box" en una pregunta y asumir que siempre significa la técnica del Ch4. Verifica qué está preguntando realmente la pregunta.

Static Testing vs. Static Analysis Ch3

- **Static testing** = la categoría amplia — evaluar un producto de trabajo *sin ejecutarlo* (incluye revisiones Y análisis basado en herramientas)
- **Static analysis** = el subconjunto basado en herramientas del static testing — análisis automatizado frente a una sintaxis formal

Trampa: "Static analysis y static testing son lo mismo." — FALSO. Static analysis es *un tipo* de static testing.

Walkthrough vs. Technical Review vs. Inspection Ch3

	Walkthrough	Technical Review	Inspection
Dirigido por	Autor	Moderador	Moderador (NO el autor)
Formalidad	Baja–Media	Media–Alta	La más alta
Escribano	No requerido	Depende	Obligatorio
¿El autor como líder?	Sí	No	Nunca

Trampa: "En un walkthrough, la revisión individual previa es obligatoria." — FALSO. Es *opcional* (el syllabus dice "no requerida"). **Trampa:** "En un walkthrough, un escribano es obligatorio." — FALSO. Un escribano es opcional en los walkthroughs; el autor puede incluso desempeñar ese rol. **Trampa:** "En una inspection, el autor puede liderar la revisión." — FALSO. El autor no puede ser el líder de la revisión ni el escribano.

Equivalence Partitioning vs. Boundary Value Analysis Ch4

- **EP** — selecciona *un valor representativo* de cada partición (cualquier valor, no necesariamente el extremo)
- **BVA** — apunta específicamente a los *extremos* de las particiones ordenadas; extiende EP, no lo reemplaza

Trampa: "BVA reemplaza EP." — FALSO. BVA extiende EP añadiendo casos de prueba específicos de límites sobre él.

2-value BVA vs. 3-value BVA Ch4

- **2 valores:** el valor límite + su vecino más cercano de la partición adyacente
- **3 valores:** el valor límite en sí + el valor inmediatamente inferior a él + el valor inmediatamente superior a él

Trampa: "3-value BVA prueba tres particiones." — FALSO. Prueba tres valores específicos alrededor de un límite.

Statement Coverage vs. Branch Coverage Ch4

- **Statement coverage:** cada sentencia ejecutable se ejecuta al menos una vez
- **Branch coverage:** cada rama (resultado verdadero/falso de cada decisión) es ejercida

Regla crítica: Branch coverage **subsume** statement coverage — el 100% de branch coverage garantiza el 100% de statement coverage. Lo inverso NO es verdadero. **Trampa:** "El 100% de statement coverage significa que todas las ramas están probadas." — FALSO.

Severity vs. Priority Ch5

- **Severity** = qué tan gravemente impacta el defecto en el sistema (visión técnica)
- **Priority** = con qué urgencia debe corregirse (visión de negocio)

Trampa: "Un defecto de alta severidad siempre es de alta prioridad." — FALSO. Ejemplo: un crash en una funcionalidad usada por el 0,1% de los usuarios = alta severidad, baja prioridad. Ejemplo: un error tipográfico en la página de inicio de sesión = baja severidad, alta prioridad.

Test Monitoring vs. Test Control ch5

- **Monitoring** = recopilar datos y verificar el progreso frente al plan (observar)
- **Control** = tomar acciones correctivas cuando se encuentran desviaciones (actuar)

Trampa: "Monitoring y control son la misma actividad." — FALSO. Monitoring sin control es solo observación.

Product Risk vs. Project Risk ch5

- **Product risk** = riesgo de que el sistema no cumpla con las expectativas de calidad → impulsa las decisiones de *prueba*
 - **Project risk** = riesgo para el cronograma, presupuesto o recursos del proyecto → gestionado por la *gestión de proyectos*
-

Trampas del Examen — Afirmaciones que Parecen Verdaderas pero No lo Son

Afirmación trampa	Por qué es incorrecta
"Las pruebas prueban que el software es correcto."	Las pruebas demuestran la <i>presencia</i> de defectos, nunca su ausencia (Principio 1)
"Con suficiente tiempo, podemos probarlo todo."	Las pruebas exhaustivas son imposibles (Principio 2)
"Si todas las pruebas pasan, el software no tiene defectos."	La ausencia de defectos es una falacia (Principio 7)
"Más pruebas siempre significa mayor calidad."	Las pruebas dependen del contexto; más no siempre es mejor (Principio 6)
"El autor puede liderar o ser el escribano de una inspección."	No está permitido — regla única para las inspecciones
"Q1 en los testing quadrants está orientado al negocio."	Q1 está orientado a la <i>tecnología</i> + apoyo al equipo; Q2 está orientado al negocio
"La regression testing solo se ejecuta después de una corrección de errores."	Cualquier cambio — mejoras, refactorizaciones, migraciones — puede romper el comportamiento existente
"El 100% de statement coverage garantiza pruebas completas."	No prueba todas las ramas; branch coverage es más robusta
"Las pruebas exploratorias no tienen estructura."	Usan <i>sesiones</i> con límite de tiempo guiadas por un <i>test charter</i>
"0-switch coverage prueba secuencias de transiciones."	0-switch = solo transiciones individuales, sin encadenamiento
"Las pruebas de caja blanca son siempre dinámicas."	El <i>análisis</i> de caja blanca puede aplicarse estáticamente a pseudocódigo o código no ejecutable
"Planning Poker es la única técnica de estimación nombrada en CTFL 4.0."	CTFL v4.0.1 nombra cuatro técnicas: Estimation based on ratios, Extrapolation, Wideband Delphi y Three-point estimation. Planning Poker es una <i>variante</i> de Wideband Delphi, no una técnica independiente.
"Un walkthrough requiere revisión individual previa."	La revisión individual es <i>opcional</i> en los walkthroughs. Un escribano también es opcional (no obligatorio).
"BVA puede aplicarse a cualquier partición."	BVA solo se aplica a particiones <i>ordenadas</i> (aquellas con valores mínimos/máximos)

Consejos para Evitar Trampas

Lee con cuidado el enunciado de la pregunta. Las preguntas del ISTQB frecuentemente dependen de una sola palabra: *MEJOR*, *MÁS adecuado*, *EXCEPTO*, *NO*, *MENOS probable*. Pasar por alto esa palabra invierte la respuesta correcta.

Conoce la diferencia entre los K-levels en la pregunta. - Una pregunta K1 te pide *identificar* o *recordar* — elige la definición correcta. - Una pregunta K2 te pide *explicar* o *distinguir* — elige la explicación más precisa. - Una pregunta K3 te da un *escenario* — aplica la técnica y deriva el resultado.

Cuando dos respuestas parecen correctas, ve por la más específica. El ISTQB prefiere la respuesta que más precisamente coincide con la definición del syllabus. La respuesta "casi correcta" suele ser un distractor que intercambia una palabra clave.

Para preguntas de técnica K3, trabájalo en papel. No intentes hacer EP, BVA o state transition en tu cabeza. Dibuja rápidamente las particiones o la tabla — detectarás errores que pasarías por alto mentalmente.

La definición del syllabus prevalece sobre la práctica de la industria. Si llevas años trabajando en QA y algo "se siente" correcto, pero contradice la definición del syllabus CTFL 4.0, elige el syllabus. El examen evalúa el estándar, no la variación del mundo real.

Presta atención a los absolutos. Palabras como *siempre*, *nunca*, *todo*, *solo*, *imposible* son frecuentemente trampas. Los principios de prueba 1 y 2 abordan específicamente qué es y qué no es posible en las pruebas.

Cuando dudes entre dos respuestas, primero elimina. Descarta las dos respuestas claramente incorrectas. Luego compara las dos restantes frente a la redacción exacta del syllabus — la respuesta correcta usará la terminología correcta.